



软件工程



课程内容提要



- 软件过程模型 (★★★★)
- 基于构件的软件工程 (★★)
- 逆向工程 (★)
- 净室软件工程 (★)
- 需求工程 (★★)
- 系统分析与设计 (★★)
- 软件测试 (★★)
- 系统运行与软件维护 (★)



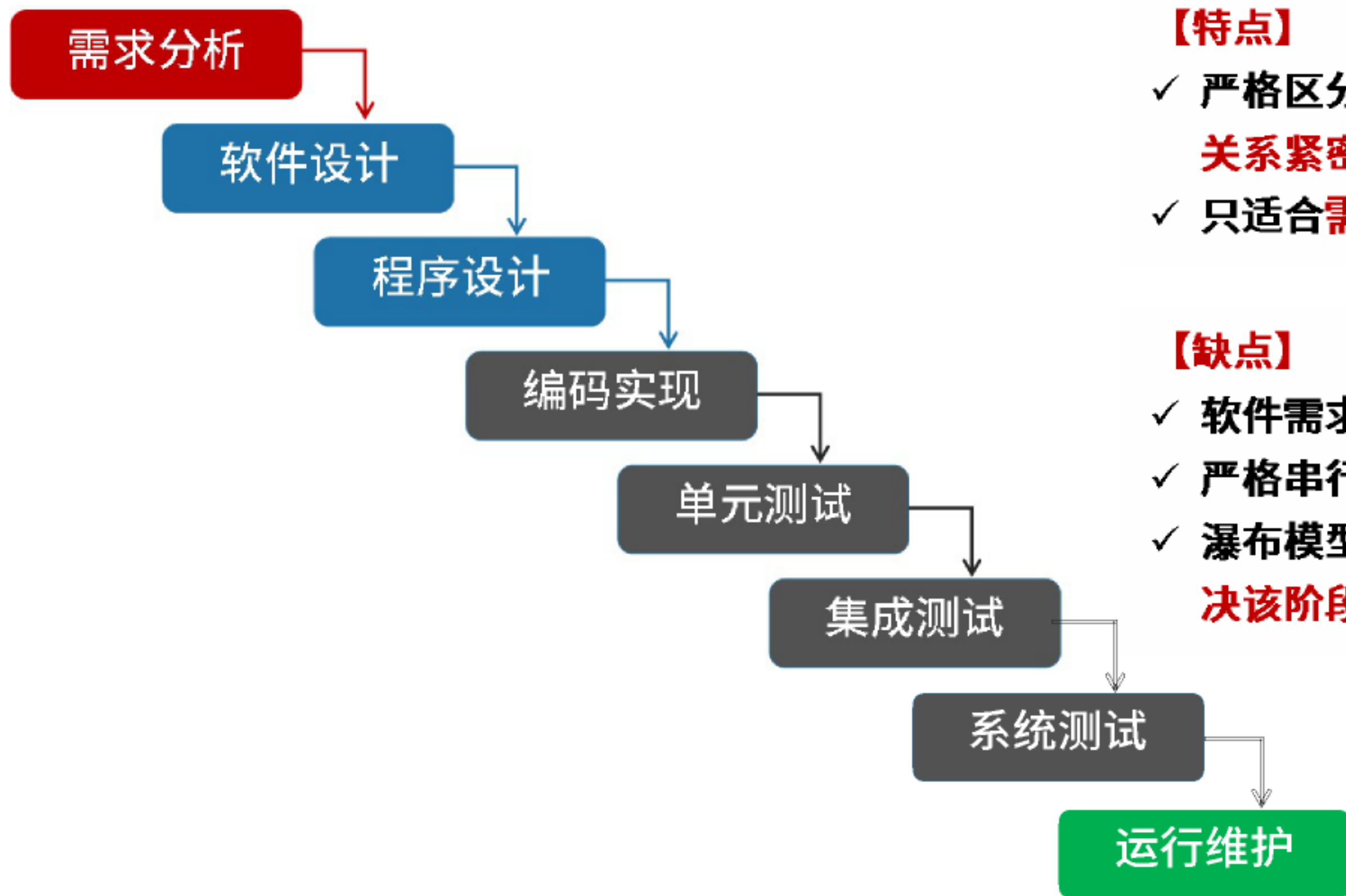
软件过程模型



- ✓ **瀑布模型**
- ✓ V模型【瀑布变种】
- ✓ **原型模型**
- ✓ **螺旋模型【原型+瀑布】**
- ✓ 构件组装模型/基于构件的开发方法
- ✓ 快速应用开发RAD【瀑布+构件组装】
- ✓ **统一过程/统一开发方法**
- ✓ **敏捷开发方法**



瀑布模型



【特点】

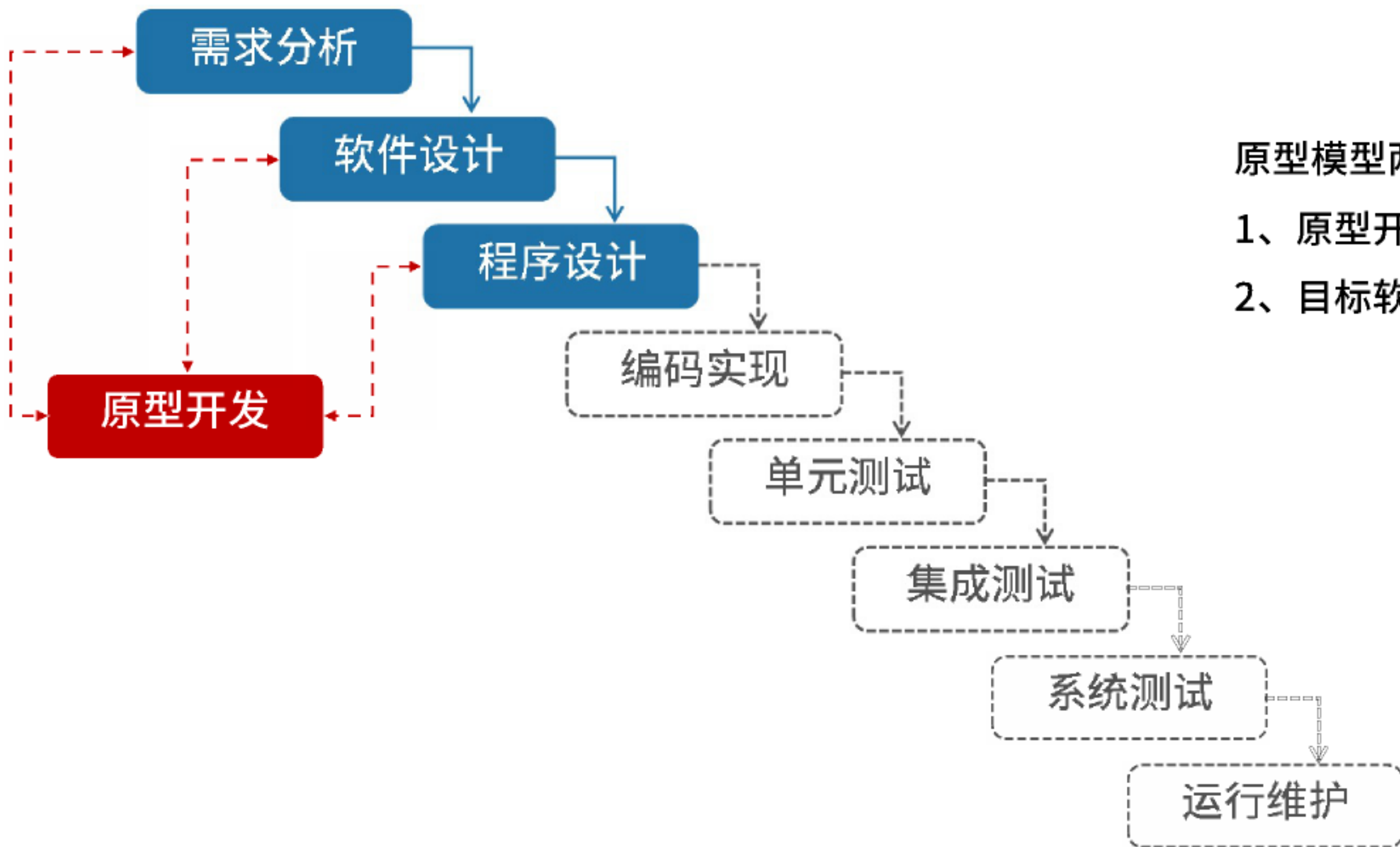
- ✓ 严格区分阶段，每个阶段因果关系紧密相连
- ✓ 只适合需求明确的项目

【缺点】

- ✓ 软件需求完整性、正确性难确定
- ✓ 严格串行化，很长时间才能看到结果
- ✓ 瀑布模型要求每个阶段一次性完全解决该阶段工作，这不现实。



原型模型

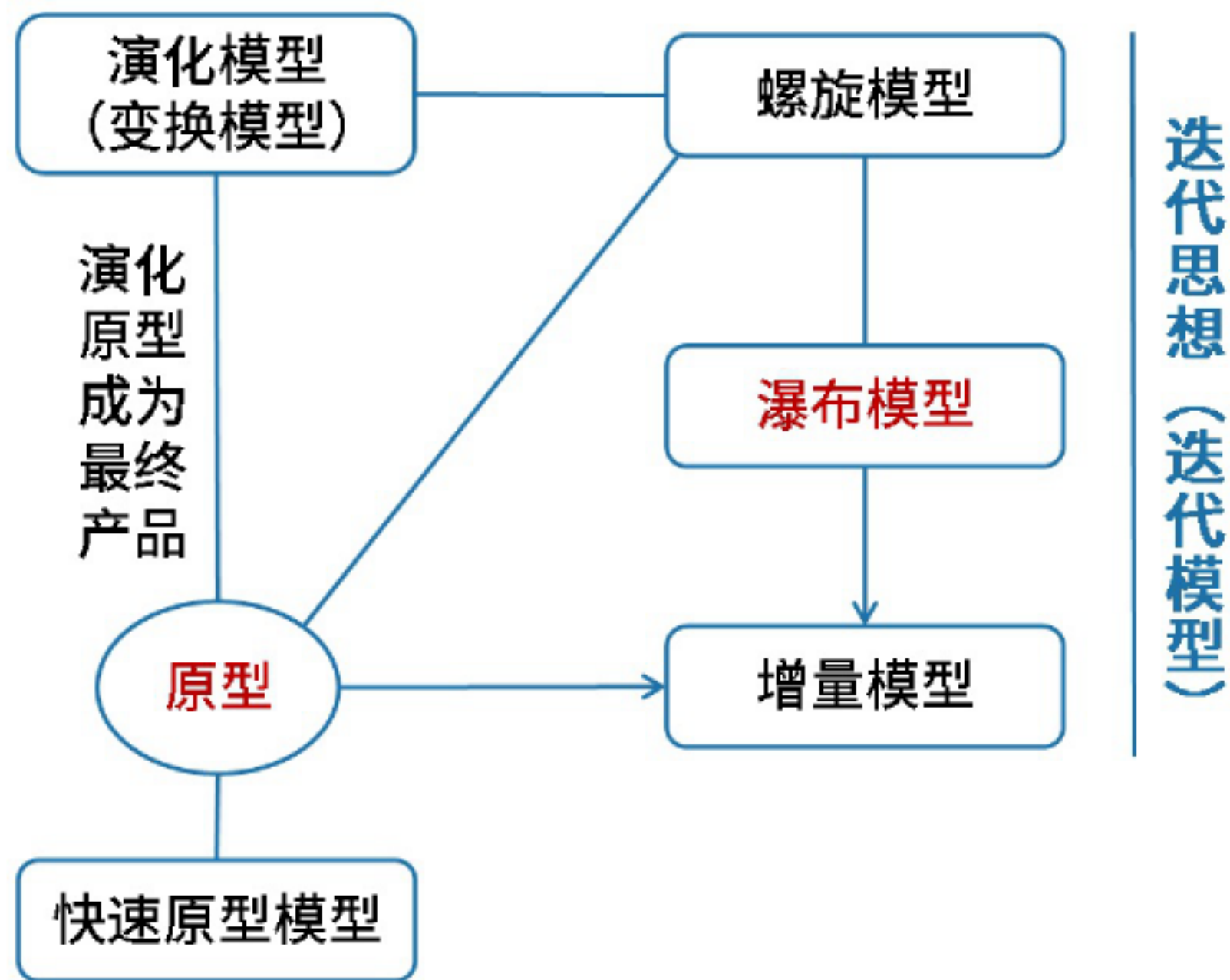


原型模型两个阶段：

- 1、原型开发阶段
- 2、目标软件开发阶段

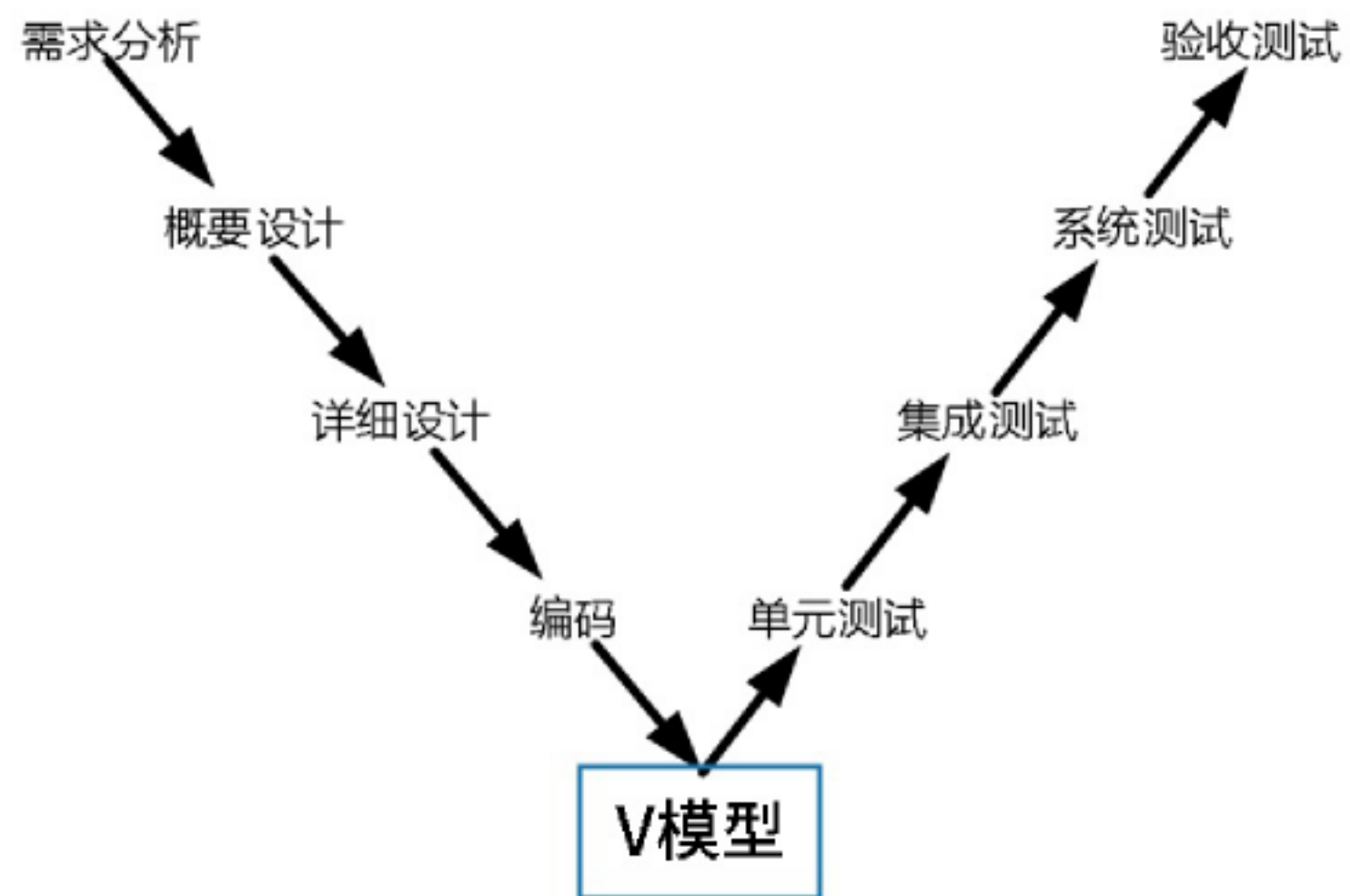


原型相关模型





V模型



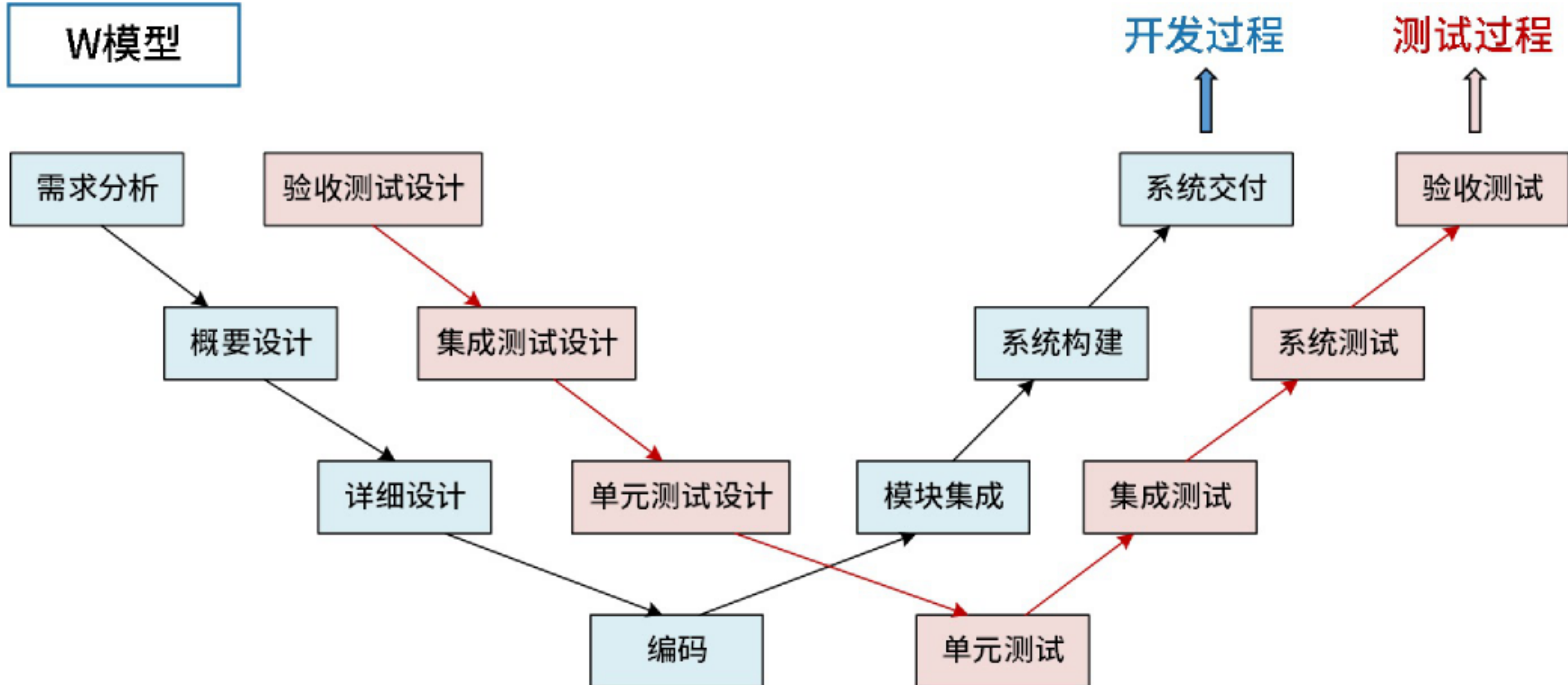
测试贯穿于始终
测试分阶段，测试计划提前



W模型



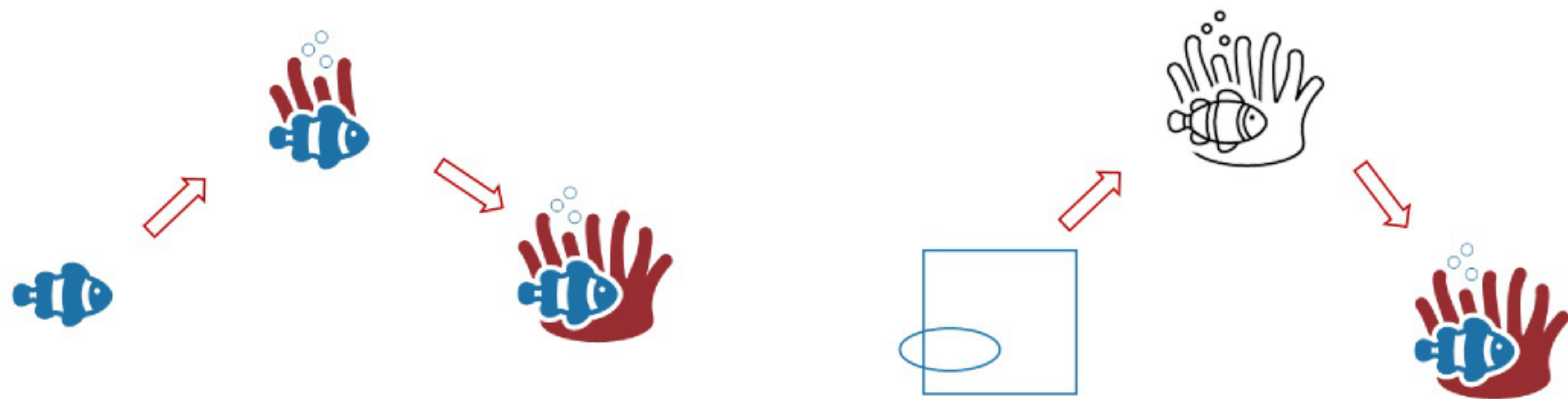
W模型



测试和开发【并行进行】



迭代与增量



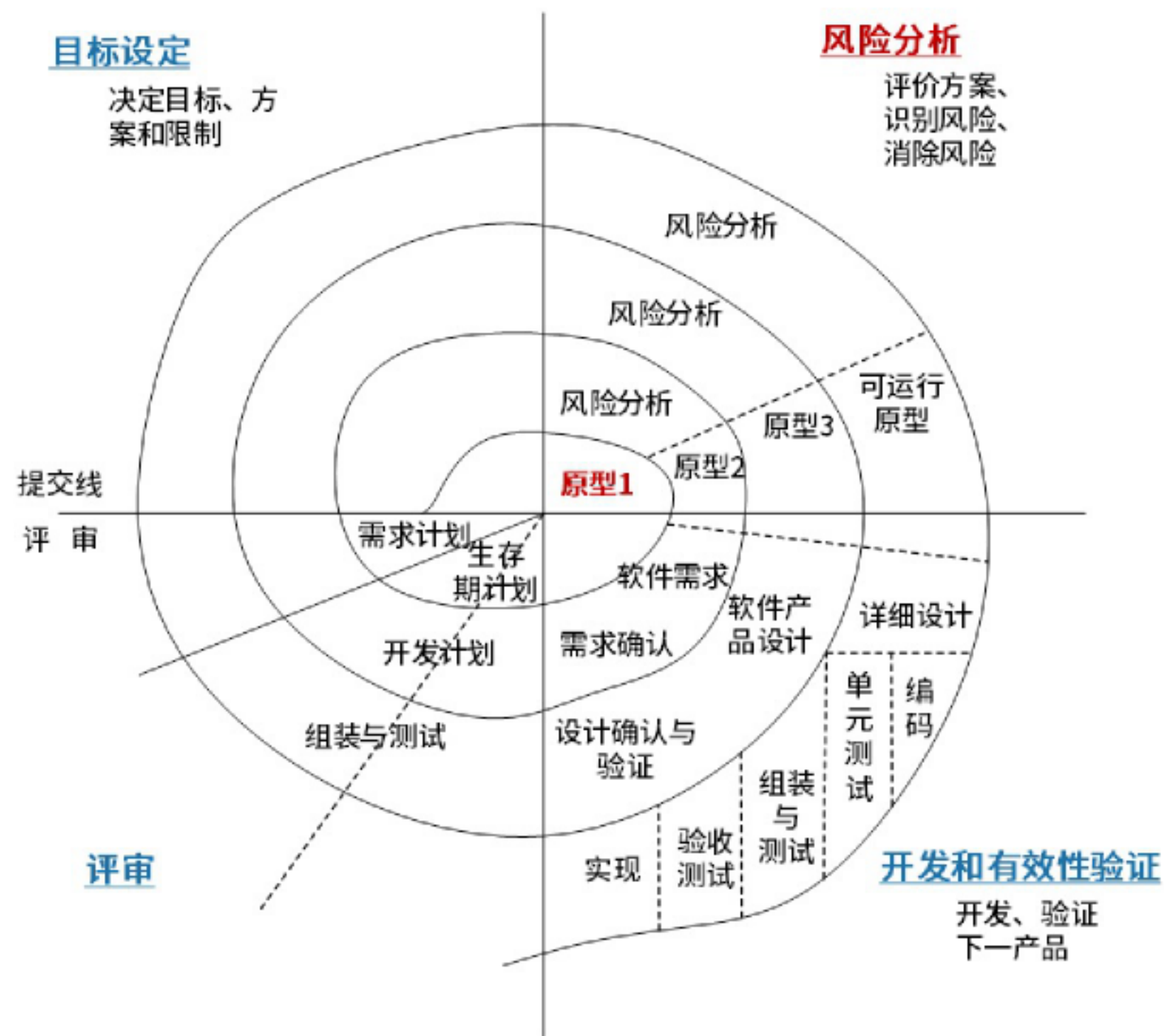
增量型

迭代型

增量与迭代
【UP / 敏捷】



螺旋模型

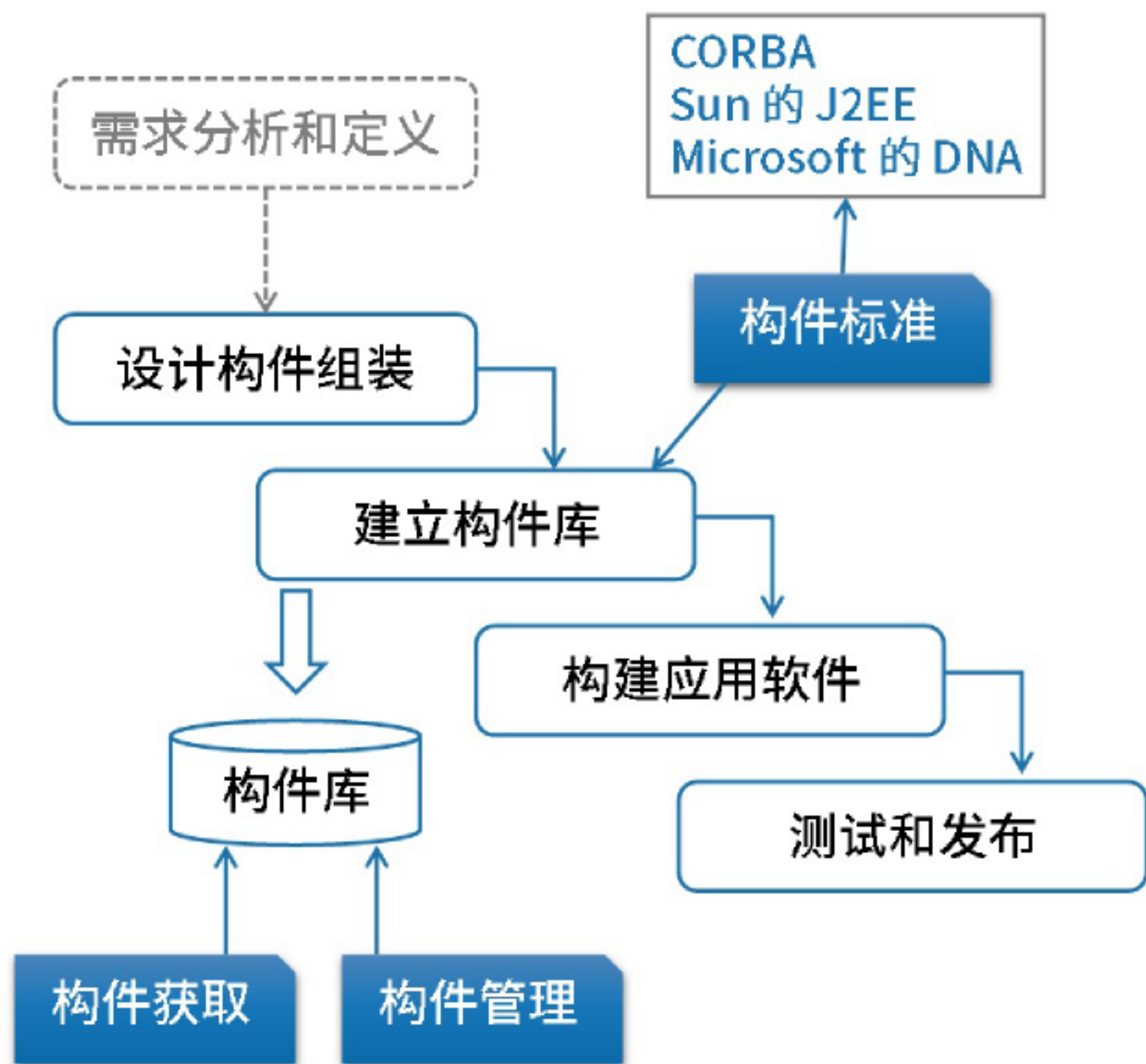


以快速原型为基础+瀑布模型

考虑了风险问题



构件组装模型



【优点】 易扩展、易重用、降低成本、安排任务更灵活。

【缺点】 构件设计要求经验丰富的架构师、设计不好的构件难重用、强调重用可能牺牲其他指标（如性能）、第三方构件质量难控制。

【示例】

方舱医院

乐高积木



基于构件的软件工程（CBSE）



CBSE体现了【购买而不是重新构造】的哲学。

【CBSE的构件应该具备的特征】

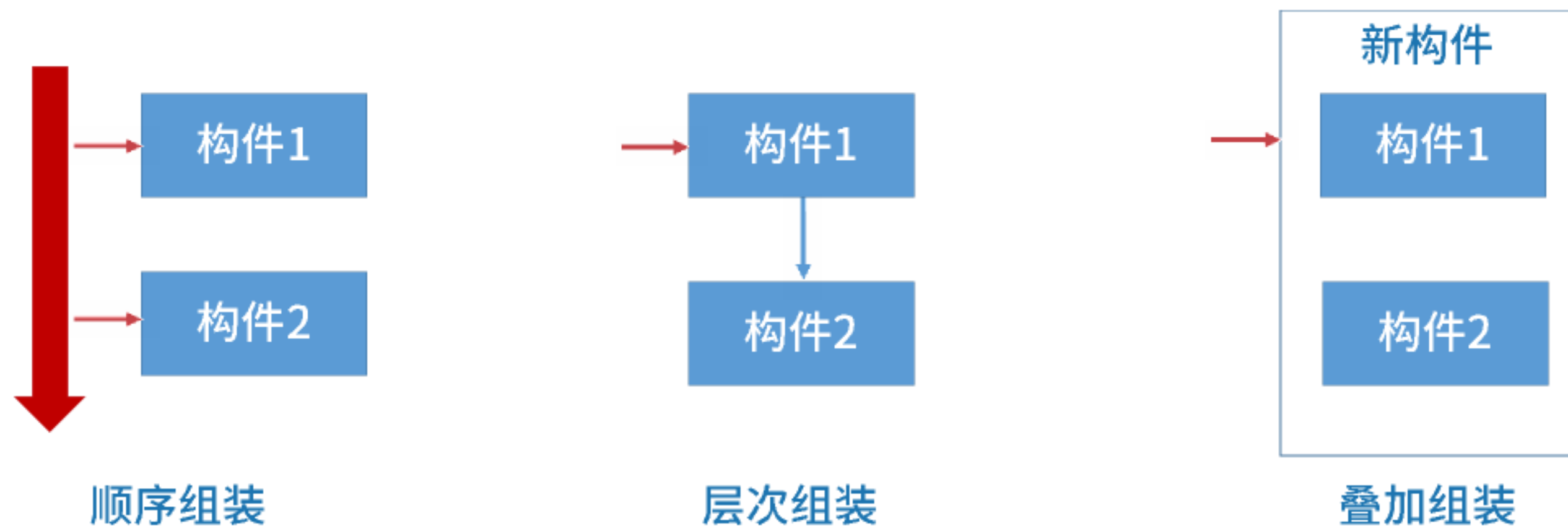
- 1、可组装性：所有外部交互必须通过公开定义的接口进行。
- 2、可部署性：构件总是二进制形式的，能作为一个独立实体在平台上运行。
- 3、文档化：用户根据文档来判断构件是否满足需求。
- 4、独立性：可以在无其他特殊构件的情况下进行组装和部署。
- 5、标准化：符合某种标准化的构件模型。



【构件的组装】（一般都要借助胶水代码）

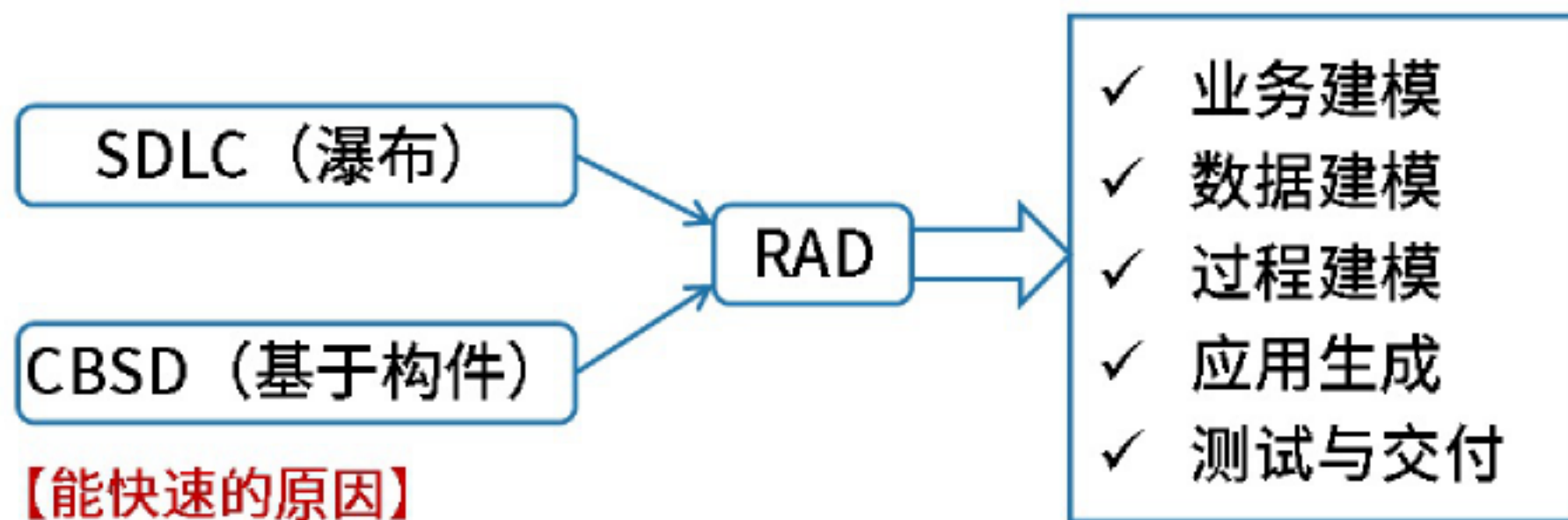
- 1、**顺序组装**：按顺序调用已经存在的构件，可以用两个已经存在的构件来创建一个新的构件。
- 2、**层次组装**：被调用构件的“提供”接口必须和调用构件的“请求”**接口兼容**。
- 3、**叠加组装**：**多个构件合并形成新构件**，新构件整合原构件的功能，**对外提供新的接口**。

组装可能出现3种不兼容：**参数不兼容、操作不兼容、操作不完备**。





快速应用开发模型 (RAD)





统一过程



- 定义最终产品视图和业务模型
- 确定系统范围

初始



细化

- 设计及确定系统架构
- 制定工作计划及资源要求

用例驱动
以架构为中心
迭代和增量



构造

- 开发剩余构件和应用程序功能，把这些构件集成为产品，并进行详细测试



移交

- 确保软件对最终用户是可用的，进行β测试，制作产品发布版本





统一过程



9个核心 workflows

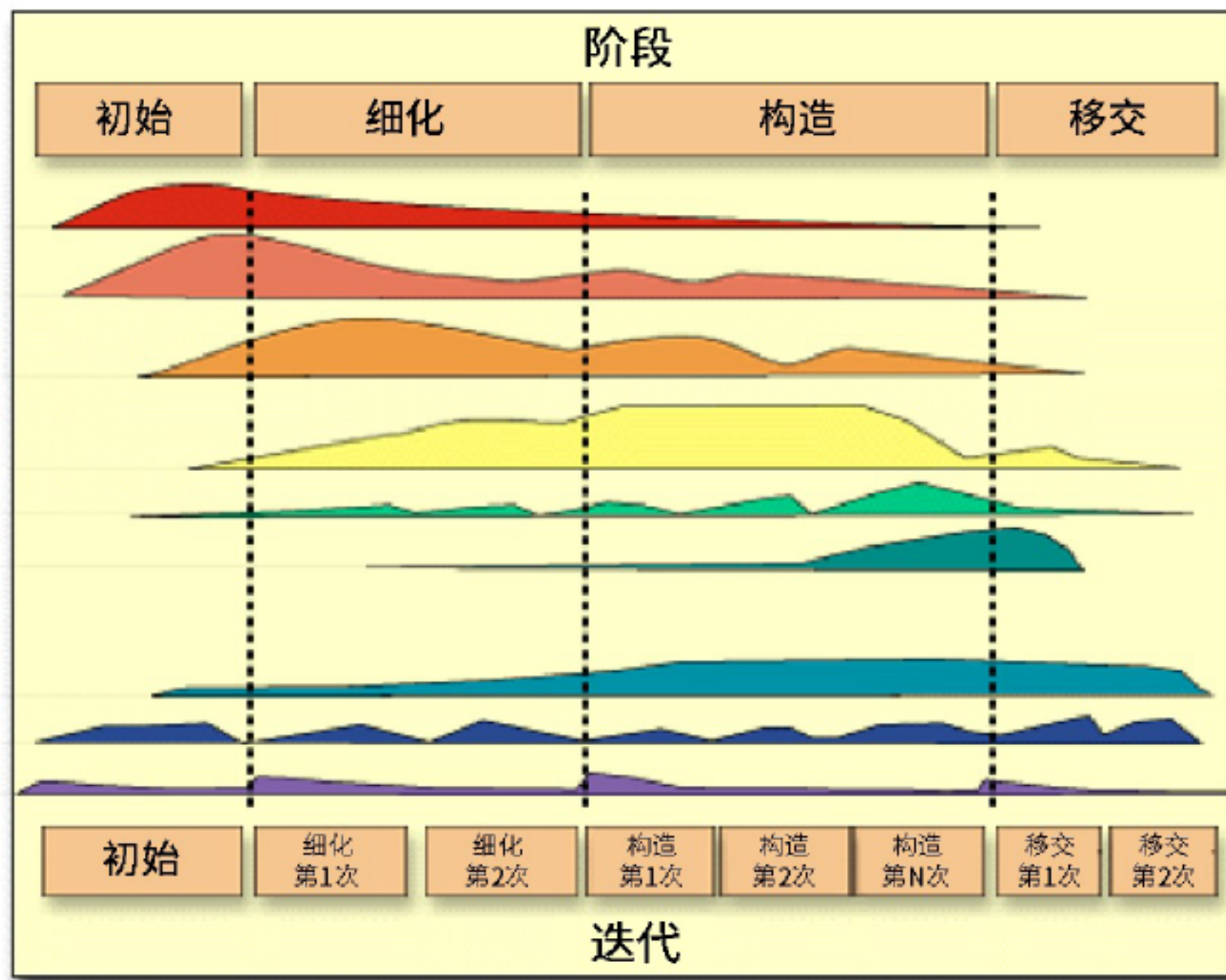
过程

业务建模
需求
分析与设计

实现
测试
部署

支持

配置与变更管理
项目管理
环境





习题讲解



() 把整个软件开发流程分成多个阶段，每一个阶段都由目标设定、风险分析、开发和有效性验证以及评审构成。

- A 原型模型
- B 瀑布模型
- C 螺旋模型
- D V模型



习题讲解



统一过程开发方法分初始、细化、构建和移交四个阶段，每次通过这四个阶段就会产生一代软件，其中（ ）阶段的核心任务是完成系统架构。统一过程开发方法从另一个维度看，有9个核心工作流，其中不包括（ ）。

A 初始

B 细化

C 构建

D 移交

A 商业建模

B 实现

C 环境

D 风险



敏捷方法



敏捷宣言

- ✓个体和交互胜过过程和工具
- ✓可工作的软件胜过大量的文档
- ✓客户合作胜过合同谈判
- ✓响应变化胜过遵循计划

如：结构化方法

无软件开发方法

无序，不可控

传统软件开发方法

敏捷方法

传统软件开发方法

预设性的

以开发过程为本

整体分阶段

敏捷方法

适应性的

以人为本

增量迭代，小步快跑

适合小型项目



敏捷方法



敏捷方法 - XP

4大价值观

沟通【加强面对面沟通】

简单【不过度设计】

反馈【及时反馈】

勇气【接受变更的勇气】

12条过程实践规则

简单设计

测试驱动

代码重构

结对编程

持续集成

现场客户

发行版本小型化

系统隐喻

代码集体所有制

规划策略

规范代码

40小时工作机制



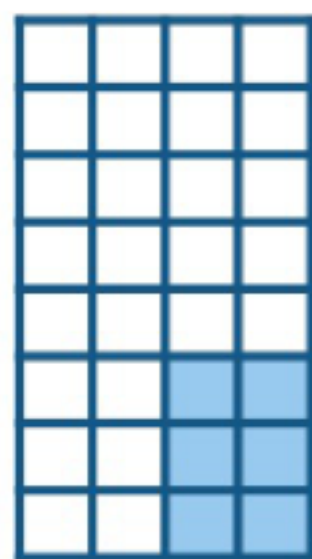
敏捷方法 (SCRUM)



从相关方处获得需求



产品负责人 (PO)



产品待办列表



团队

团队选择该迭代
承诺完成多少工作

迭代计划会议



迭代待办事项

下个迭代版本
产品待办列表整理



敏捷教练
(Scrum Master)



迭代
(1-4周)

在一个迭代版本中
目标没有发生变化



每日站会



迭代评审会议



潜在可发布的产品增量



迭代回顾会议



敏捷方法



- ◆ **极限编程 (XP)**：价值观【交流、朴素、反馈、勇气】、近螺旋式的开发方法。
- ◆ **水晶方法**：提倡“**机动性**”的方法，拥有对不同类型项目非常有效的敏捷过程。
- ◆ **SCRUM**：侧重于项目管理。
- ◆ **特征驱动开发方法 (FDD)**：认为有效的软件开发需要3要素【**人、过程、技术**】。定义了6种关键的项目角色：**项目经理、首席架构设计师、开发经理、主程序员、程序员和领域专家**。
- ◆ **开放式源码**：程序开发人员在**地域上分布很广**【其他方法强调集中办公】。
- ◆ **ASD方法**：其核心是三个非线性的、重叠的开发阶段：猜测、合作与学习。
- ◆ **动态系统开发方法 (DSDM)**：倡导以业务为核心。



习题讲解



下列关于敏捷方法的叙述中，错误的是（ ）。

- A 与传统方法相比，敏捷方法比较适合需求变化大或者开发前期对需求不是很清晰的项目
- B 敏捷方法尤其适合于开发团队比较庞大的项目
- C 敏捷方法的思想是适应性，而不是预设性
- D 敏捷方法以原型开发思想为基础，采用迭代式增量开发



习题讲解



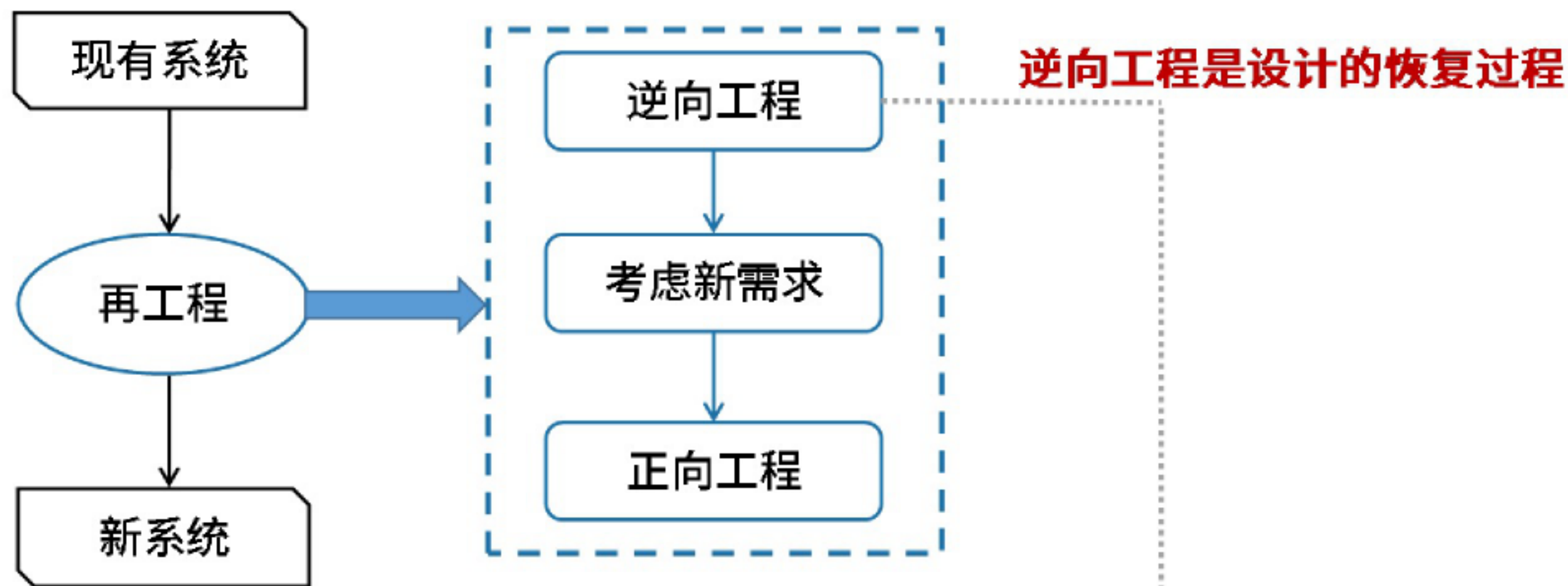
() 提倡“机动性”的方法，其家族实际上是一组经过证明、对不同类型项目非常有效的敏捷过程。() 认为有效的软件开发需要3要素：人、过程、技术。

- A 水晶系列 (Crystal) 开发方法
- B 开放式源码 (Open Source) 开发方法
- C SCRUM开发方法
- D 特征驱动开发方法 (FDD)

- A 自适应软件开发 (ASD)
- B 极限编程 (XP) 开发方法
- C 开放统一过程开发方法 (Open UP)
- D 特征驱动开发方法 (FDD)



逆向工程



实现级：包括程序的**抽象语法树、符号表、过程**的设计表示

结构级：包括反映**程序分量之间相互依赖关系**的信息，例如调用图、结构图、程序和数据结构

功能级：包括反映**程序段功能及程序段**之间关系的信息，例如数据和控制流模型

领域级：包括反映程序分量或程序诸实体与**应用领域概念之间对应关系**的信息，例如实体关系模型



逆向工程



与逆向工程相关的概念有重构、设计恢复、再工程和正向工程。

(1) 重构/重组 (Restructuring)。重构是指在【同一抽象级别】上【转换系统描述形式】。

(2) 设计恢复 (Design recovery)。设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

(3) 逆向工程 (Reverse engineering)：逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是设计的恢复过程。

(4) 正向工程 (Forward engineering)。正向工程是指不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

(5) 再工程/重构工程 (Re-engineering)。再工程是对现有系统的重新开发过程，包括逆向工程、新需求的考虑过程和正向工程三个步骤。



习题讲解



软件逆向工程就是分析已有的程序，寻求比源代码更高级的抽象表现形式。在逆向工程导出信息的四个抽象层次中，（ ）包括反映程序各部分之间相互依赖关系的信息；（ ）包括反映程序段功能及程序段之间关系的信息。

A 实现级

B 结构级

C 功能级

D 领域级

A 实现级

B 结构级

C 功能级

D 领域级



净室软件工程



- ✓ 净室即无尘室、洁净室。也就是一个**受控污染级别的环境**。
- ✓ 使用盒结构规约（或形式化方法）进行分析和设计建模，并且**强调将正确性验证，而不是测试**，作为发现和消除错误的主要机制。
- ✓ 使用统计的测试来获取认证被交付的软件的可靠性所必需的出错率信息。



【技术手段】

- ✓ 统计过程控制下的增量式开发：**控制迭代**
- ✓ 基于函数的规范和设计：盒子结构

定义3种抽象层次：行为视图（**黑盒**）->有限状态机视图（**状态盒**）->过程视图（**明盒**）

- ✓ **正确性验证：净室工程的核心**
- ✓ **统计测试和软件认证**：使用统计学原理，总体太大时必须采用**抽样方法**



净室软件工程



【缺点】

- ✓ 太理论化，正确性验证的步骤比较困难且耗时。
- ✓ 开发小组不进行传统的模块测试，这是不现实的。
- ✓ 脱胎于传统软件工程，不可避免带有传统软件工程的一些弊端。



需求工程



- ✓ 需求开发 (★★★★)
- ✓ 需求管理 (★★)
- ✓ UML4+1视图 (★★)
- ✓ UML图 (★★★★)
- ✓ UML关系 (★★)



需求工程阶段划分



- 软件需求是指**用户对系统**在功能、行为、性能、设计约束等方面的**期望**。

【需求工程主要活动的阶段划分】

- | | |
|--|------|
| <ul style="list-style-type: none">✓ 需求获取✓ 需求分析✓ 形成需求规格【形成SRS】✓ 需求确认与验证【形成需求基线（经过评审的SRS）】 | 需求开发 |
| <ul style="list-style-type: none">✓ <u>需求管理</u>【变更控制、版本控制、需求跟踪、需求状态跟踪】 | |

↓
需求管理是对【**需求基线**】进行管理



习题讲解

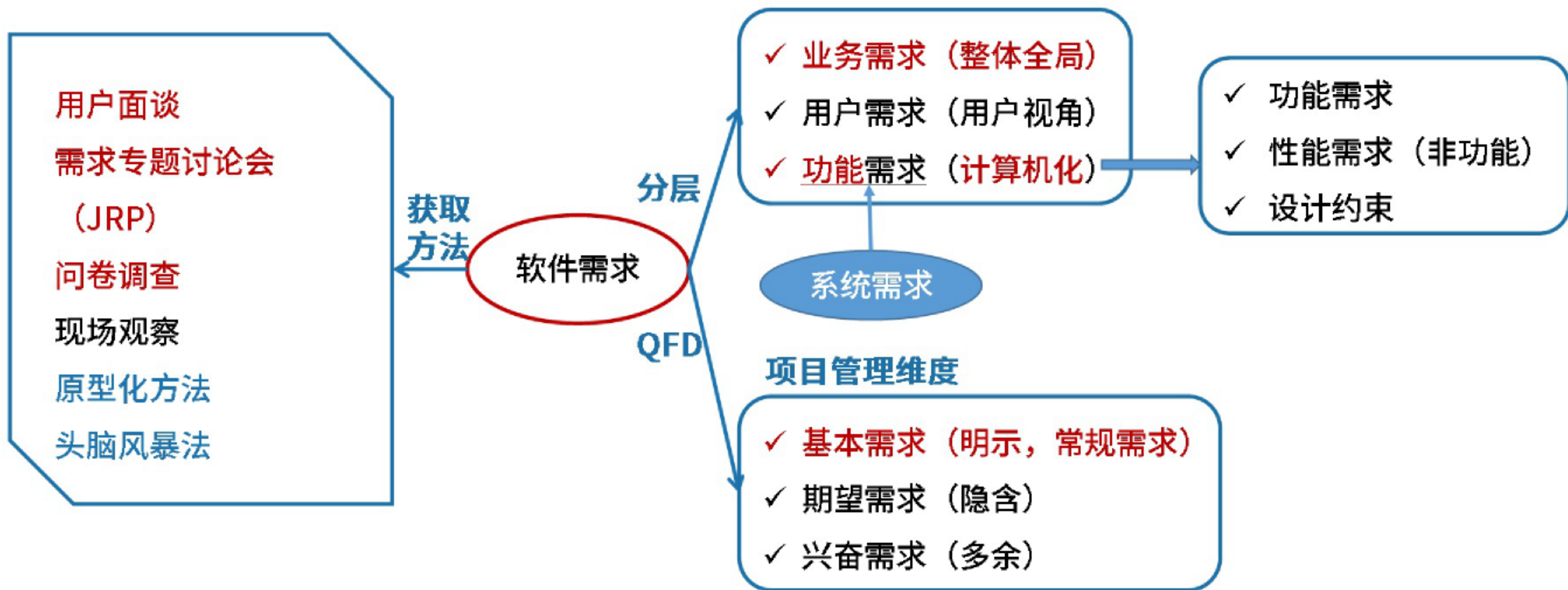


下列关于软件需求管理或需求开发的叙述中，正确的是（ ）。

- A 所谓需求管理是指对需求开发的管理
- B 需求管理包括：需求获取、需求分析、需求定义和需求验证
- C 需求开发是将用户需求转化为应用系统成果的过程
- D 在需求管理中，要求维持对用户原始需求和所有产品构件需求的双向跟踪



需求获取





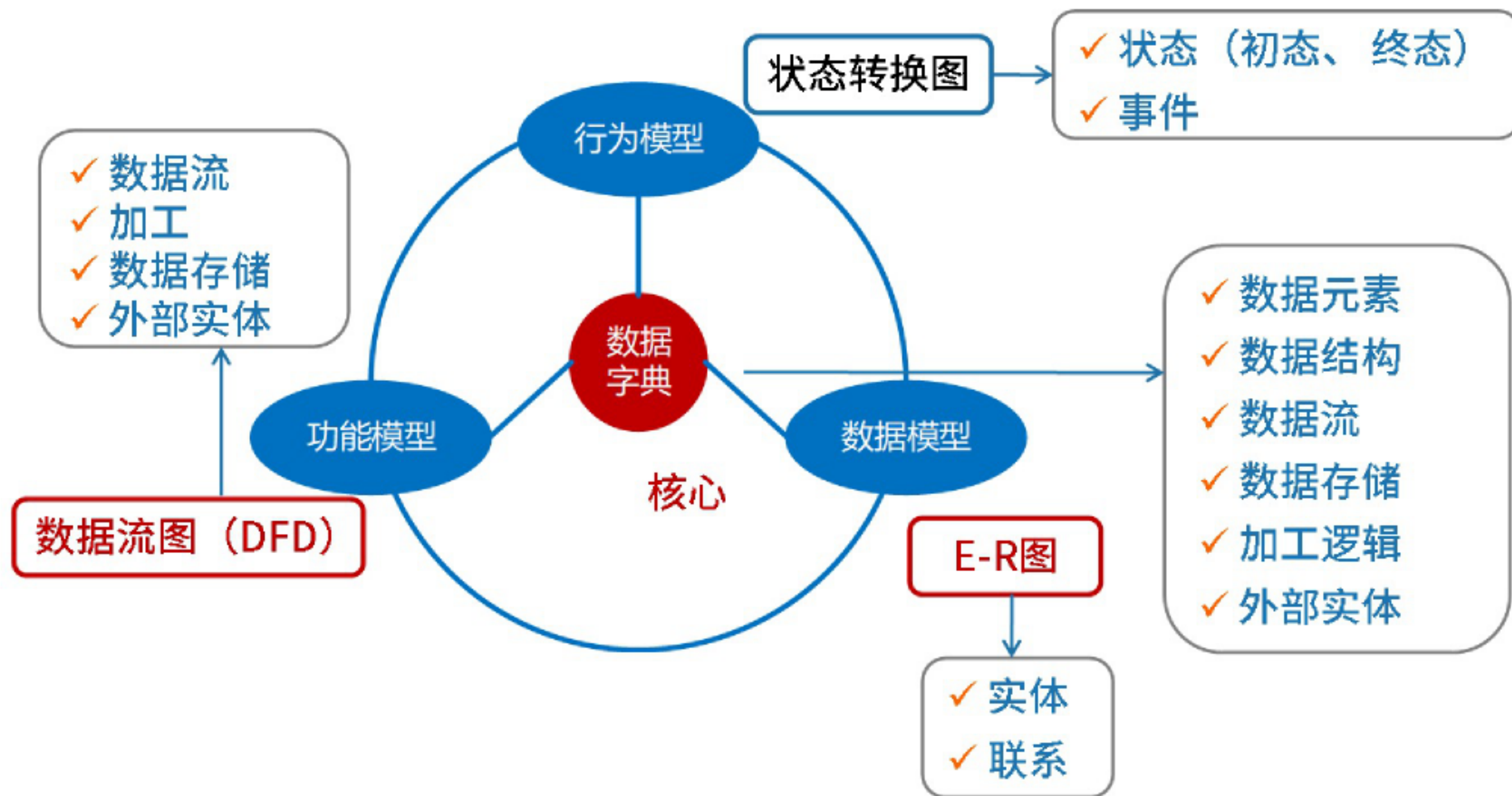
需求获取



方法	特点
用户面谈	1对1-3，有代表性的用户，了解主观想法，交互好。 成本高，要有领域知识支撑。
联合需求计划（JRP）	高度组织的群体会议，各方参与，了解想法，消除分歧，交互好，成本高。
问卷调查	用户多，无法一一访谈，成本低。
现场观察	针对较为 复杂的流程 和操作。
原型化方法	通过简易系统方式 解决早期需求不确定问题。
头脑风暴法	一群人围绕新业务， 发散思维 ，不断产生新的观点。

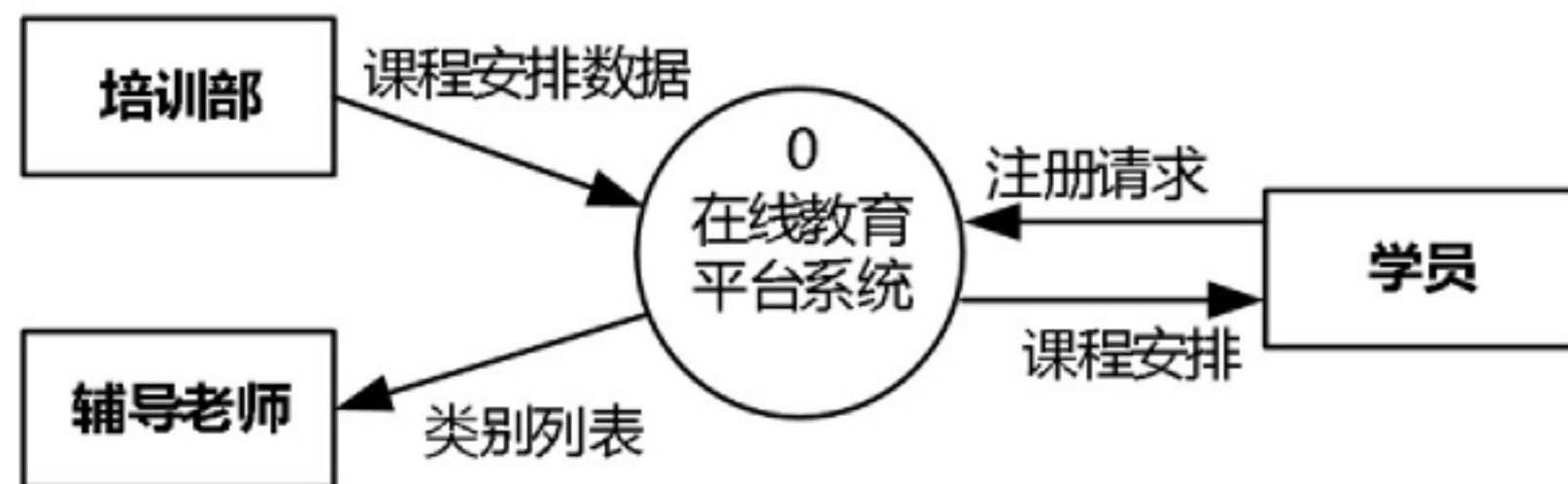


需求分析

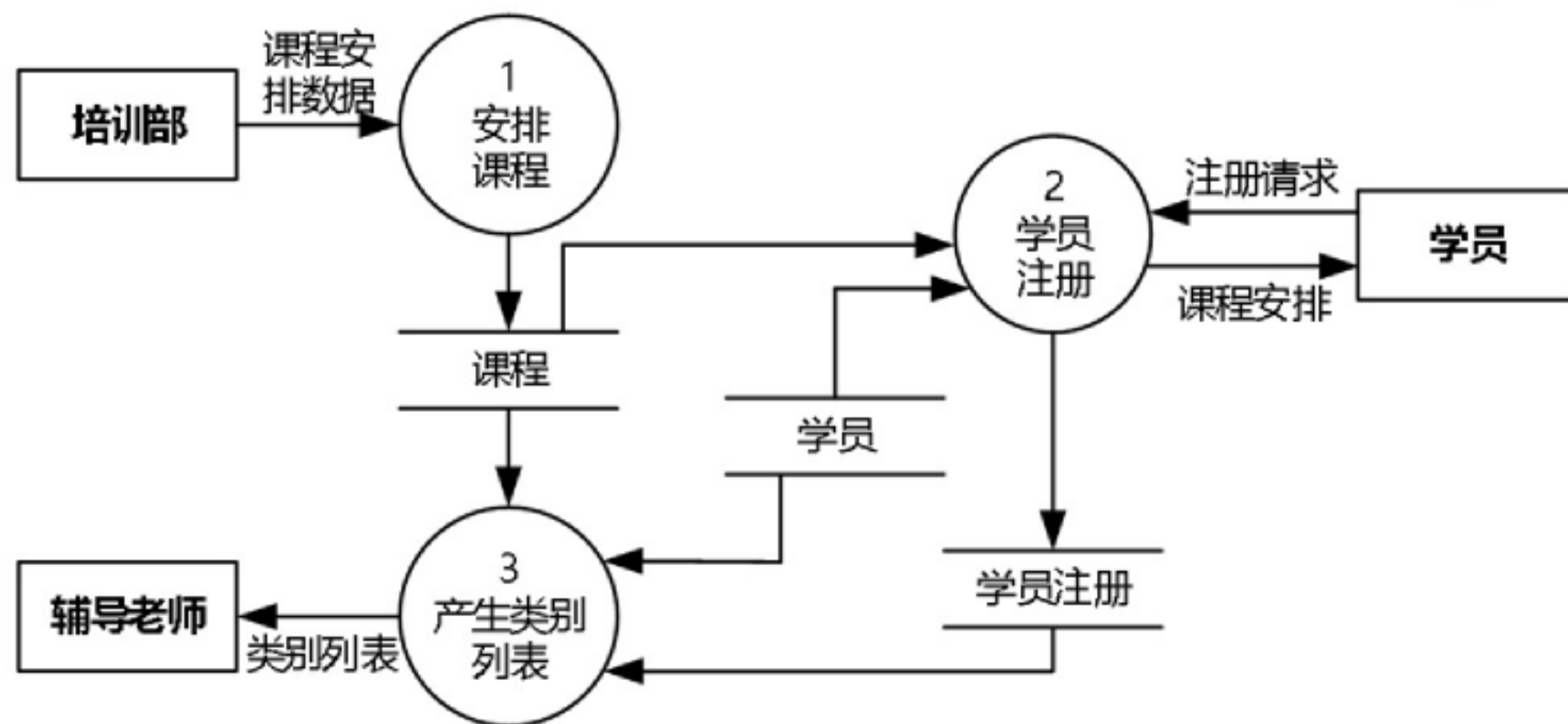




需求分析 (DFD)

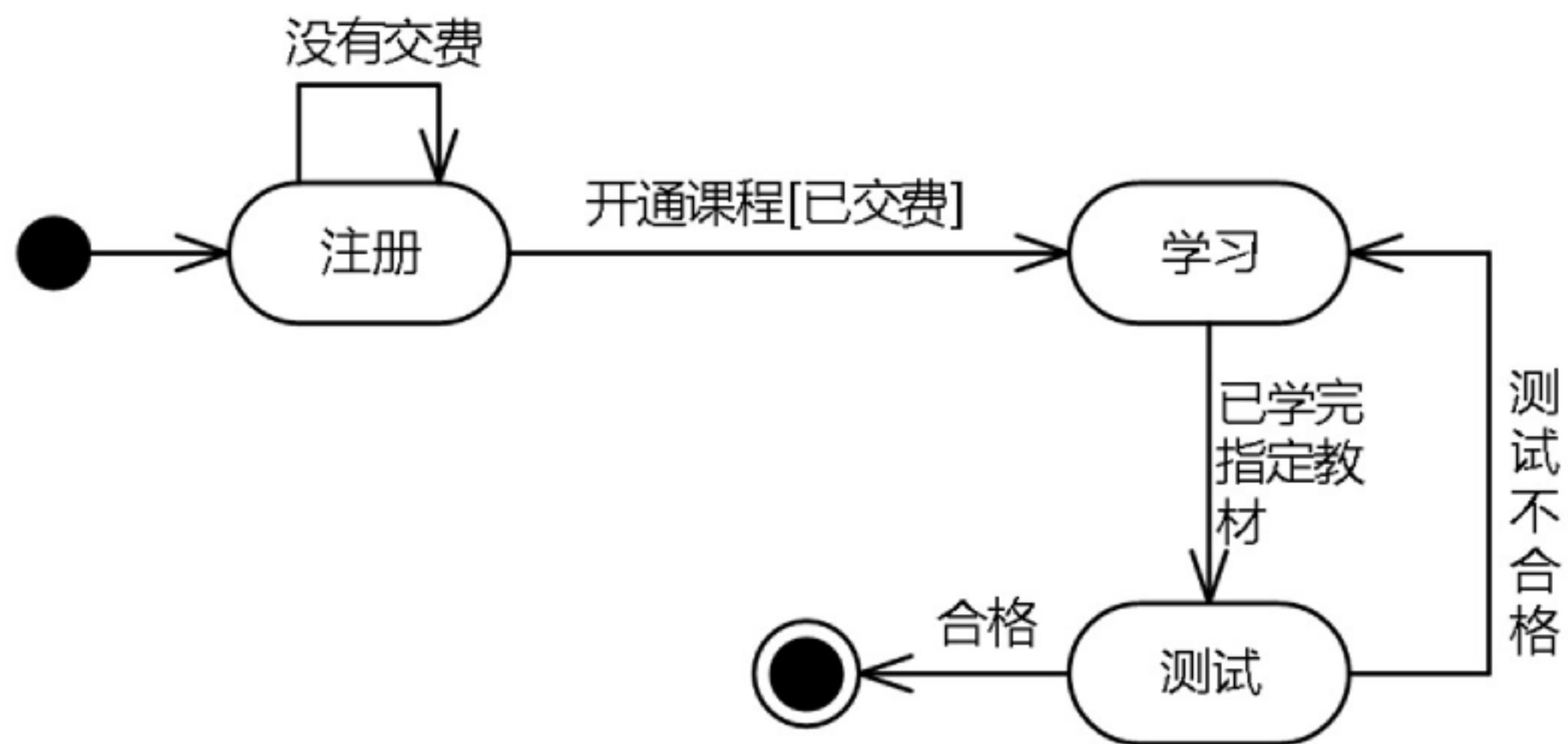


- ✓ 数据流
- ✓ 加工
- ✓ 数据存储
- ✓ 外部实体



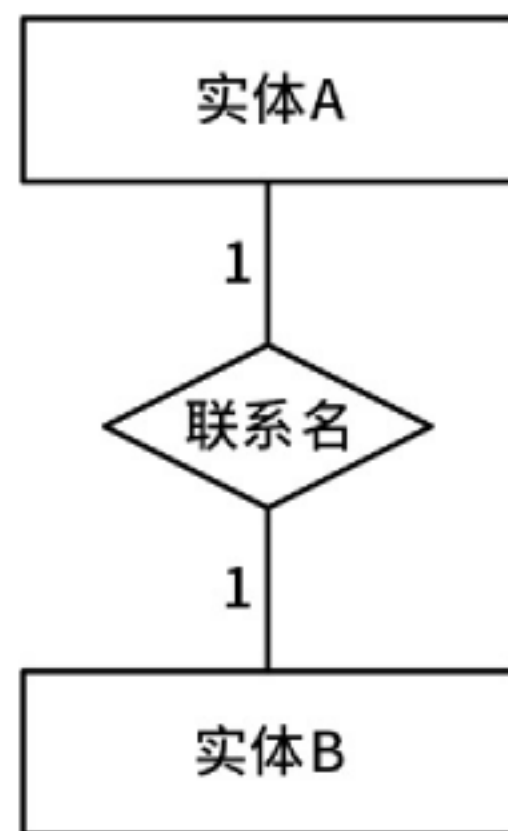


需求分析 (STD)

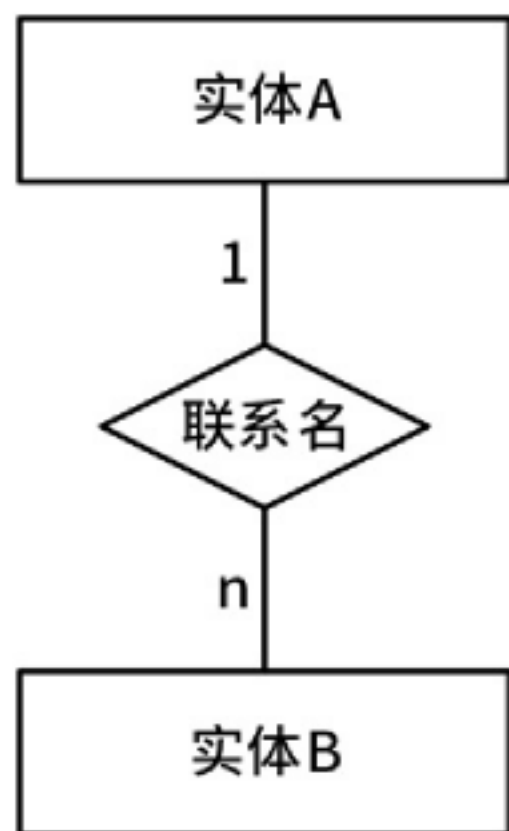




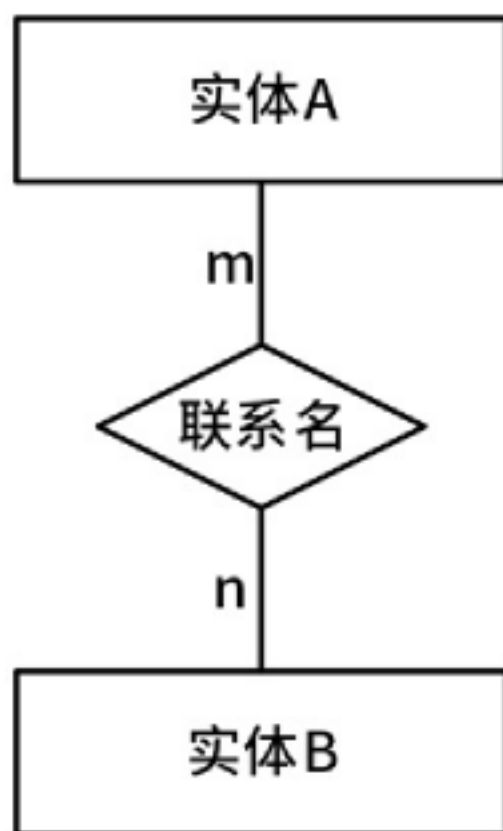
需求分析 (ER)



(a) 1: 1的联系



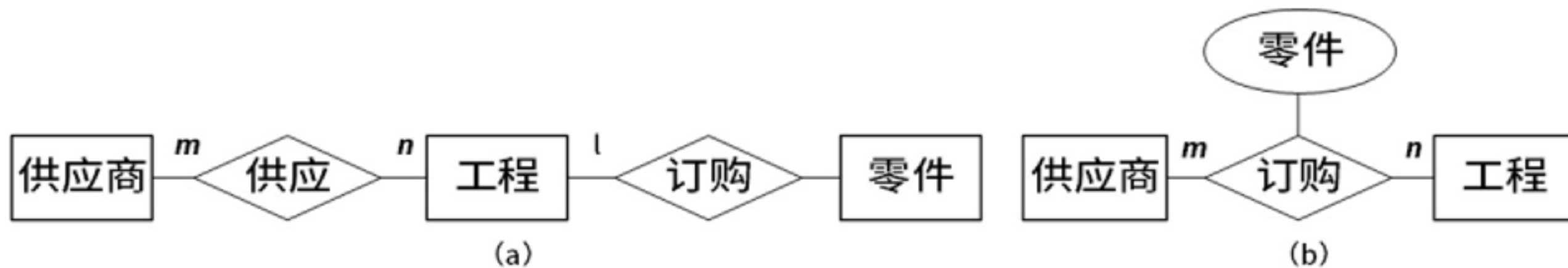
(b) 1: n的联系



(c) m: n的联系



(d) 同一实体集内一对多的联系



(a)

(b)



习题讲解



在结构化分析方法中，用（ ）表示功能模型，用（ ）表示行为模型。

A ER图

B 用例图

C DFD

D 对象图

A 通信图

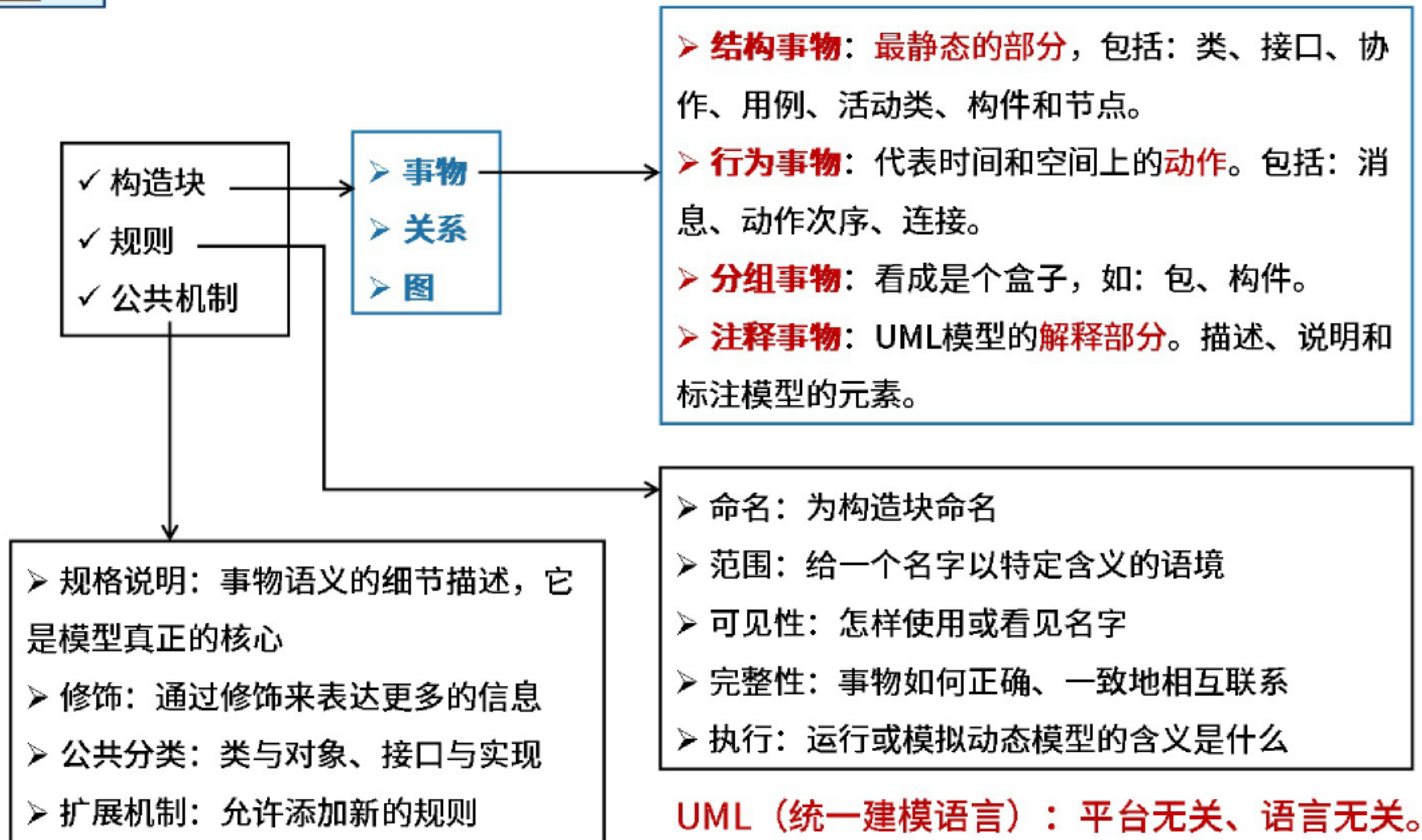
B 顺序图

C 活动图

D 状态转换图



需求分析 (UML)





需求分析 (UML)



UML图

静态图 (结构图)

动态图 (行为图)

- 类图：一组类、接口、协作和它们之间的关系
- 对象图：一组对象及它们之间的关系
- 构件图：一个封装的类和它的接口
- 部署图：软硬件之间映射
- 制品图：系统的物理结构
- 包图：由模型本身分解而成的组织单元，以及它们之间的依赖关系
- 组合结构图

- 用例图：系统与外部参与者的交互
- 顺序图：强调按时间顺序
- 通信图 (协作图)
- 定时图：强调实际时间
- 交互概览图
- 活动图：类似程序流程图，并行行为
- 状态图：状态转换变迁

交互图



需求定义（形成需求规格）



严格定义法

- ✓ 所有需求都能够被预先定义
- ✓ 开发人员与用户之间能够准确而清晰地交流
- ✓ 采用图形/文字可以充分体现最终系统

原型法

- ✓ 并非所有的需求都能在开发前被准确的说明
- ✓ 项目参加者之间通常都存在交流上的困难
- ✓ 需要实际的、可供用户参与的系统模型
- ✓ 有合适的系统开发环境
- ✓ 反复是完全需要和值得提倡的，需求一旦确定，就应遵从严格的方法



习题讲解

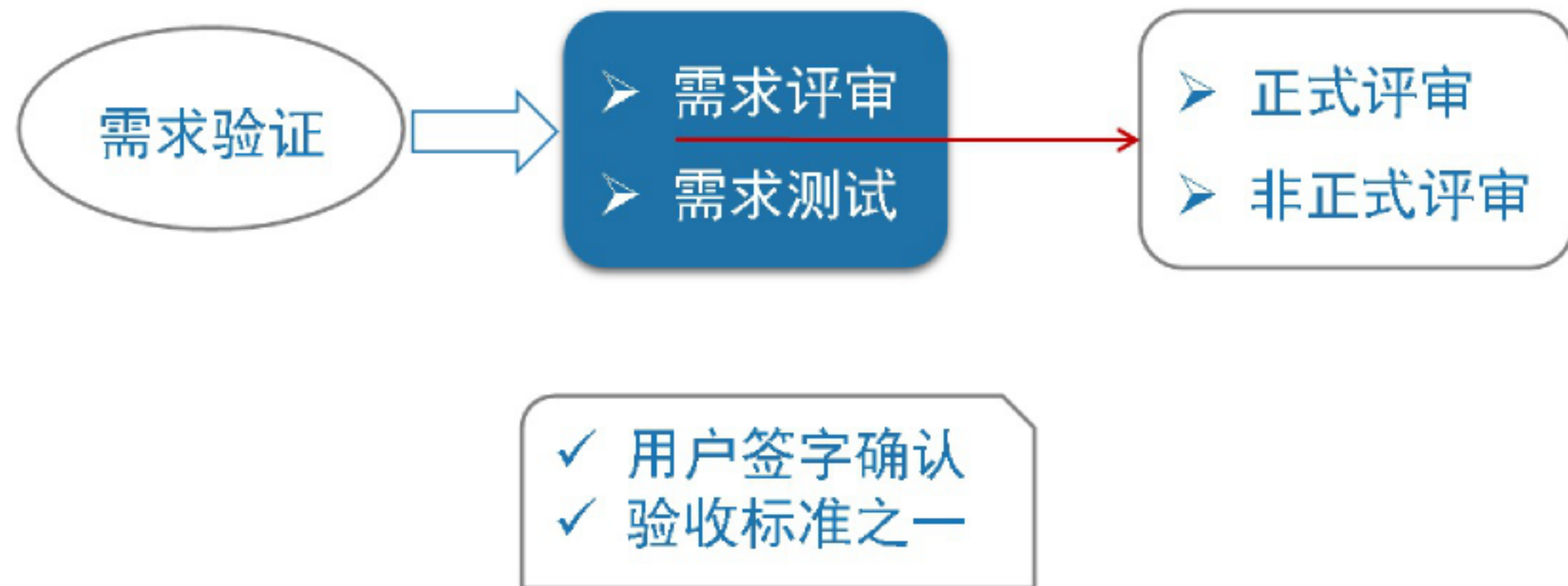


通常有两种常用的需求定义方法：严格定义方法和原型方法。下述的各种假设条件中，“（ ）”不适合使用严格定义方法进行需求定义。

- A 所有需求都能够被预先定义
- B 开发人员与用户之间能够准确而清晰地交流
- C 需求不能在系统开发前被完全准确的说明
- D 采用图形（或文字）充分体现最终系统

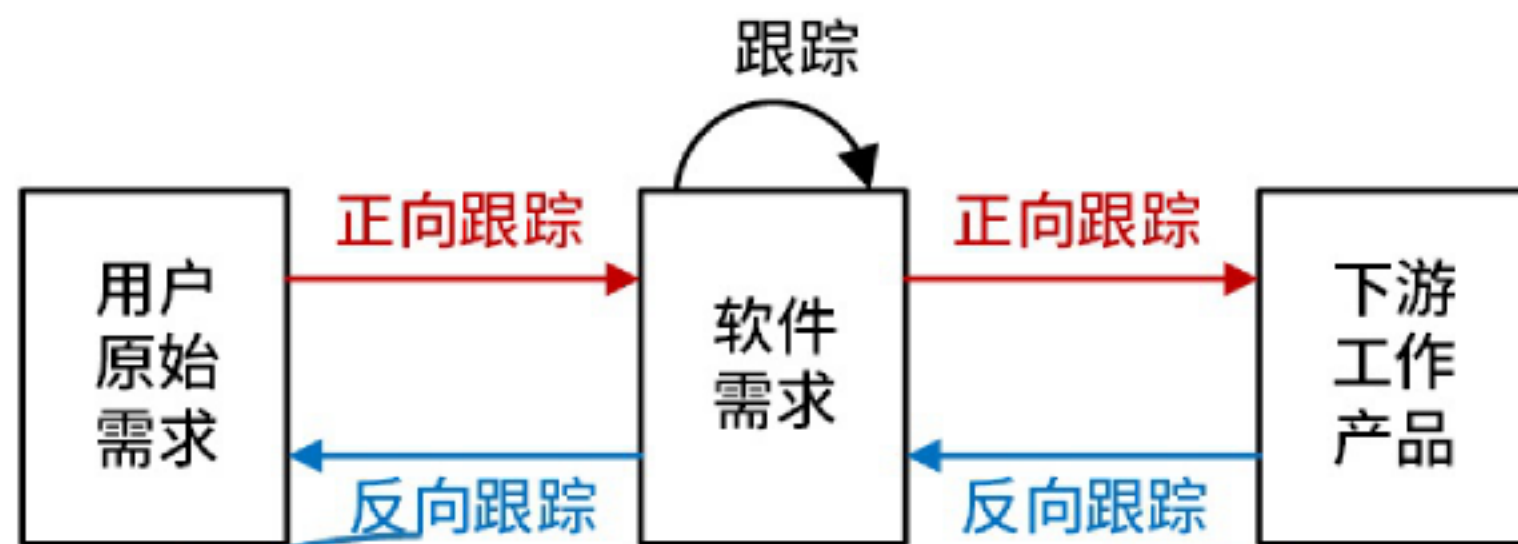


需求确认与验证





需求跟踪

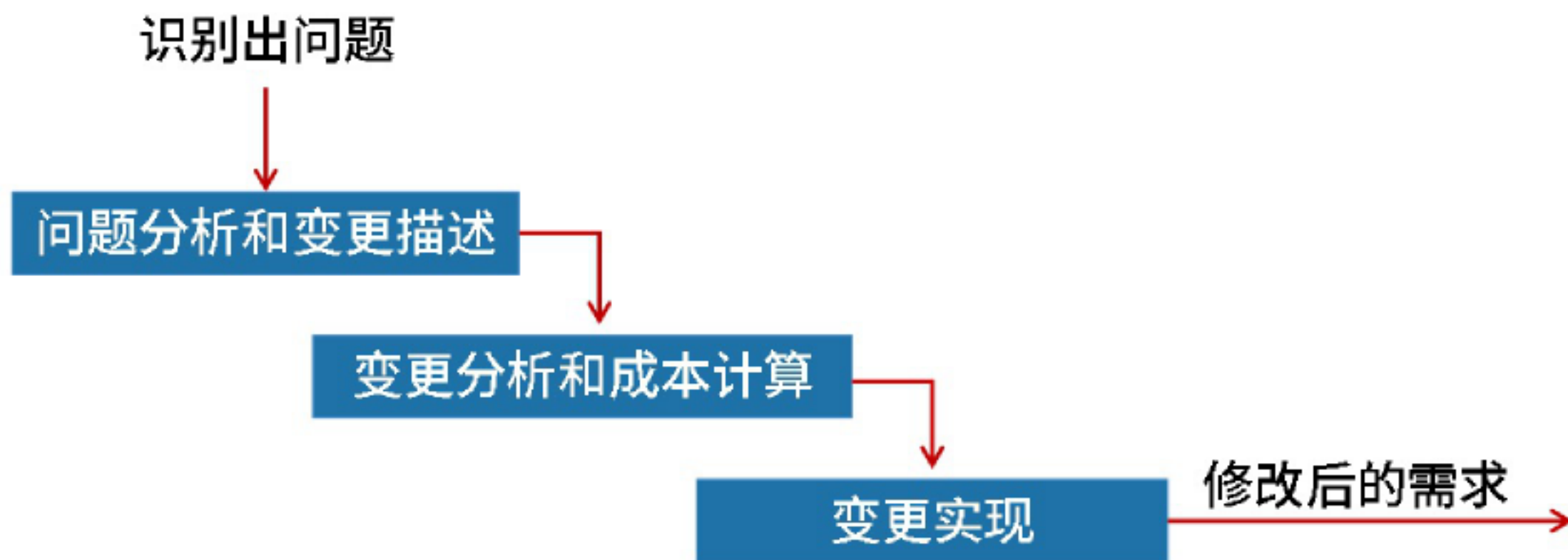


原始需求 \ 用例	UC-1	UC-2	UC-3		UC-n
FR-1	✓				
FR-2			✓		
.....					
FR-m					

用例 \ 元素	功能点	设计元素	代码模块	测试用例
UC-1				
UC-2				
.....				
UC-n				



需求变更管理过程



需求变更控制流程的十大步骤

- | | | |
|----------------|-------------------|-----------------|
| 1、明确问题 | 4、 评估 变更影响 | 7、明确解决方案 |
| 2、书面 申请 | 5、判断变更的紧急级别 | 8、 审批 管理 |
| 3、判断变更需求类别 | 6、沟通确认 | 9、 执行 变更 |
| | | 10、 版本控制 |



习题讲解



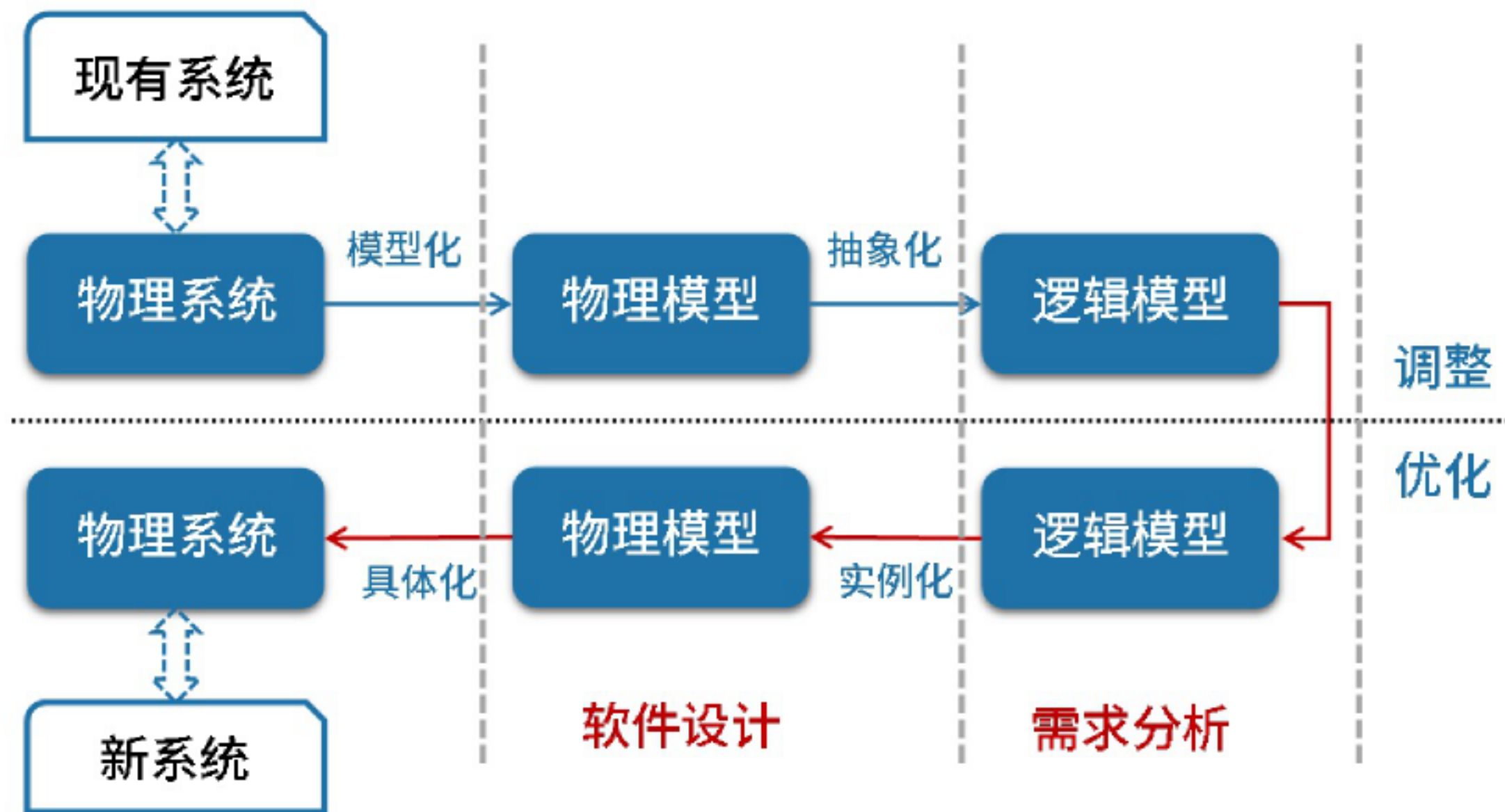
需求变更管理是需求管理的重要内容。需求变更管理的过程主要包括问题分析和变更描述、
()、变更实现。具体来说，在关于需求变更管理的描述中，() 是不正确的。

- A 变更调研
- B 变更判定
- C 变更定义
- D 变更分析和成本计算

- A 需求变更要进行控制，严格防止因失控而导致项目混乱，出现重大风险
- B 需求变更对软件项目开发有利无弊
- C 需求变更通常按特定的流程进行
- D 在需求变更中，变更审批由 CCB 负责审批



软件系统建模





软件系统建模



论软件系统建模方法及其应用

软件系统建模（Software System Modeling）是软件开发中的重要环节，通过构建软件系统模型可以帮助系统开发人员理解系统、抽取业务过程和管理系统的复杂性，也可以方便各类人员之间的交流。软件系统建模是在系统需求分析和系统实现之间架起的一座桥梁，系统开发人员按照软件系统模型开发出符合设计目标的软件系统，并基于该模型进行软件的维护和改进。

请围绕“论软件系统建模方法及其应用”论题，依次从以下三个方面进行论述。

1. 概要叙述你参与的软件系统开发项目以及你所担任的主要工作。
2. 说明软件系统开发中常用的建模方法有哪几类？阐述每种方法的特点及其适用范围。
3. 详细说明你所参与的软件系统开发项目中，采用了哪些软件系统建模方法，具体实施效果如何。



软件系统建模



1、结构化建模方法

结构化建模方法是以过程为中心的技术，可用于分析一个现有的系统以及定义新系统的业务需求。结构化建模方法所绘制的模型称为数据流图（DFD）。对于流程较为稳定的系统可考虑结构化建模方法。

2、信息工程建模方法（或数据库建模方法）

信息工程建模方法是一种以数据为中心，但过程敏感的技术，它强调在分析和研究过程需求之前，首先研究和分析数据需求。信息工程建模方法所创建的模型被称为实体联系图（ERD）。主要用于数据建模。

3、面向对象建模方法

面向对象建模方法将“数据”和“过程”集成到被称为“对象”的结构中，消除了数据和过程的人为分离现象。面向对象建模方法所创建的模型被称为对象模型。随着面向对象技术的不断发展和应用，形成了面向对象的建模标准，即UML（统一建模语言）。UML定义了几种不同类型的模型图，这些模型图以对象的形式共建一个信息系统或应用系统，是目前比较常用的建模方法。



系统设计



- ✓ 界面设计 (★★)
- ✓ 结构化设计 (★★)
- ✓ 面向对象设计 (★★★)



人机界面设计（黄金三法则）



★ 置于用户控制之下

- 以不强迫用户进入不必要的或不希望的动作的方式来定义交互方式
- 提供灵活的交互
- 允许用户交互可以被中断和撤销
- 当技能级别增加时可以使交互流水化并允许定制交互
- 使用户隔离内部技术细节
- 设计应允许用户和出现在屏幕上的对象直接交互

★ 减少用户的记忆负担

- 减少对短期记忆的要求
- 建立有意义的缺省
- 定义直觉性的捷径
- 界面的视觉布局应该基于真实世界的隐喻
- 以不断进展的方式揭示信息

★ 保持界面的一致性

- 允许用户将当前任务放入有意义的语境
- 在应用系列内保持一致性
- 如过去的交互模型已建立起了用户期望，除非有迫不得已的理由，不要改变它



习题讲解



下列关于用户界面设计的叙述中，错误的是（ ）。

- A 界面交互模型应经常进行修改
- B 界面的视觉布局应该尽量与真实世界保持一致
- C 所有可视信息的组织需要按照统一的设计标准
- D 确保用户界面操作和使用的一致性



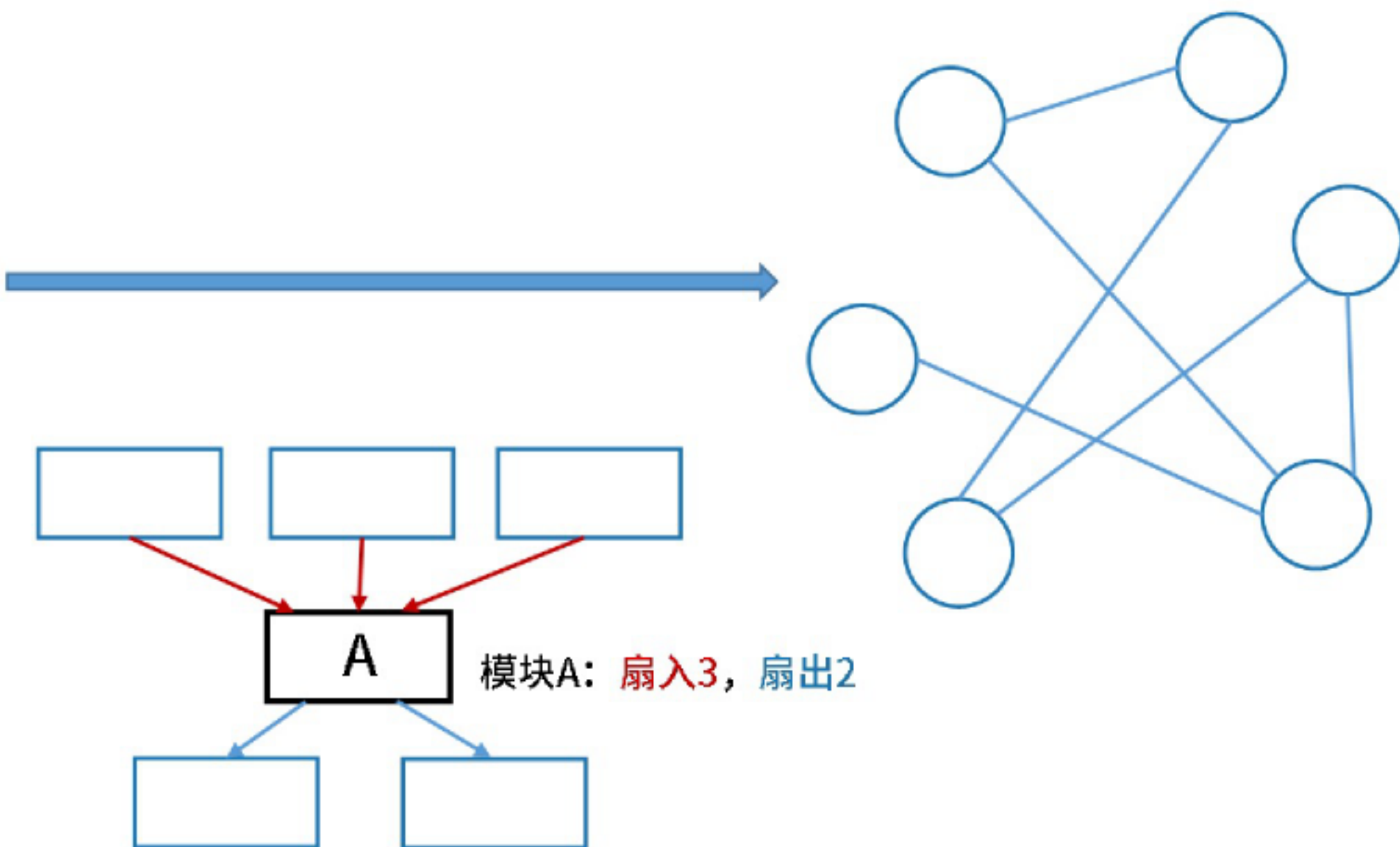
结构化设计



- **概要设计【外部设计】**：功能需求分配给软件模块，确定每个模块的功能和**调用关系**，形成**模块结构图**
- **详细设计【内部设计】**：为每个**具体任务**选择适当的技术手段和**处理方法**

结构化设计原则：

- **模块独立性原则（高内聚、低耦合）**
- 保持模块的大小适中
- 多扇入，少扇出
- 深度和宽度均不宜过高





结构化设计（内聚）



高内聚



低内聚

内聚类型	描 述
功能内聚	完成一个 单一功能 ，各个部分协同工作，缺一不可
顺序内聚	处理元素相关，而且 必须顺序执行
通信内聚	所有处理元素集中在一个数据结构的区域上
过程内聚	处理元素相关，而且必须按特定的次序执行
时间内聚 (瞬时内聚)	所包含的任务必须在 同一时间间隔 内执行
逻辑内聚	完成逻辑上相关的一组任务
偶然内聚 (巧合内聚)	完成一组没有关系或松散关系的任务



结构化设计（耦合）



低耦合



高耦合

耦合类型	描 述
非直接耦合	两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的
数据耦合	一组模块借助参数表传递简单数据
标记耦合	一组模块通过参数表传递记录信息（数据结构）
控制耦合	模块之间传递的信息中包含用于控制模块内部逻辑的信息
外部耦合	一组模块都访问同一全局简单变量，而且不是通过参数表传递该全局变量的信息
公共耦合	多个模块都访问同一个公共数据环境
内容耦合	一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口



结构化设计（模块四要素）



模块的四个要素

输入和输出：模块的输入来源和输出去向都是同一个调用者，即一个模块从调用者那儿取得输入，进行加工后再把输出返回调用者。

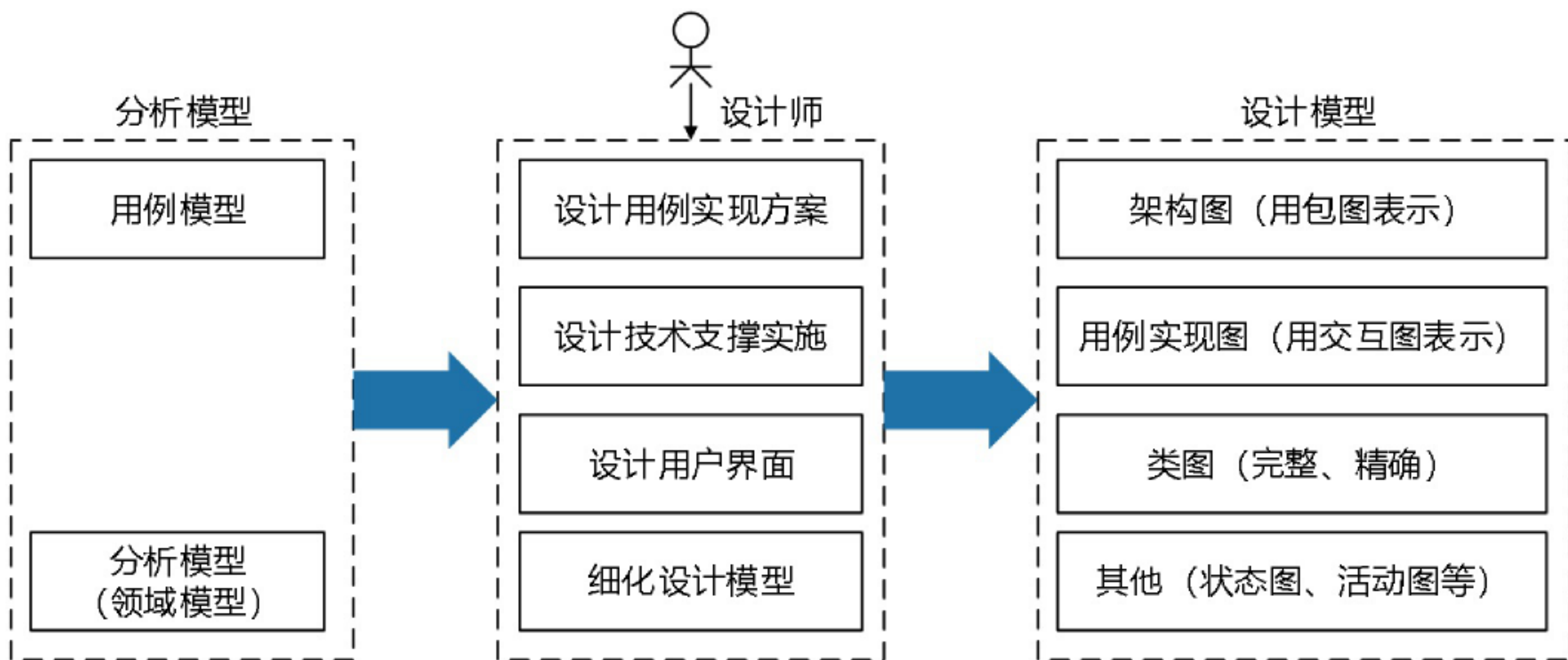
处理功能：指模块把输入转换成输出所做的工作。

内部数据：指仅供该模块本身引用的数据。

程序代码：指用来实现模块功能的程序。



面向对象设计（基本过程）





面向对象设计（类的分类）



类的分类

边界类

机器接口

API接口

人机交互

用户界面



例：显示屏、窗体、打印机接口、通信协议、对话框、菜单、购物车、报表、二维码

控制类

应用逻辑

业务逻辑

数据访问逻辑

例：身份验证器

实体类

数据

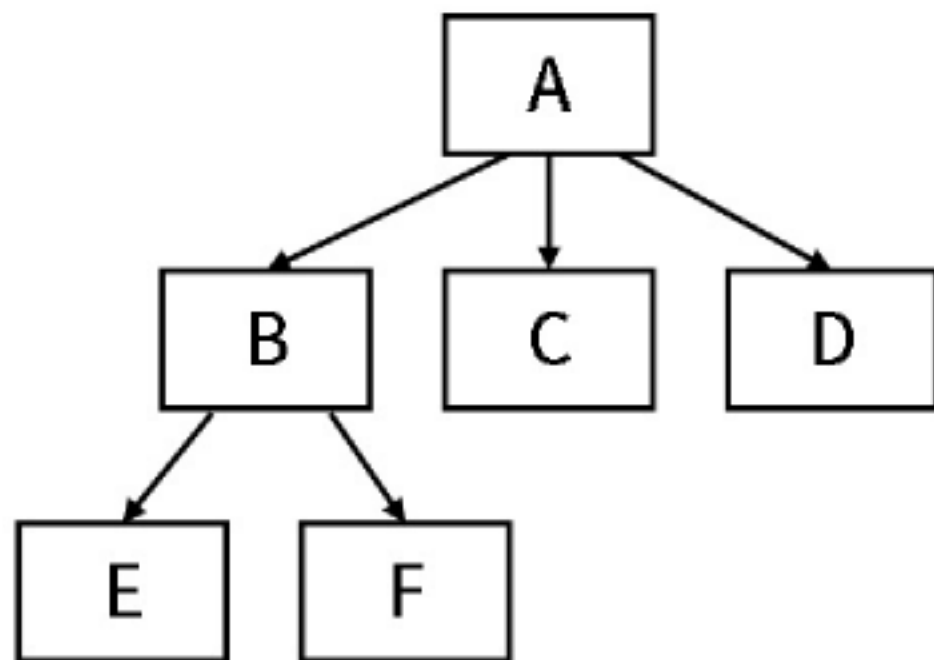
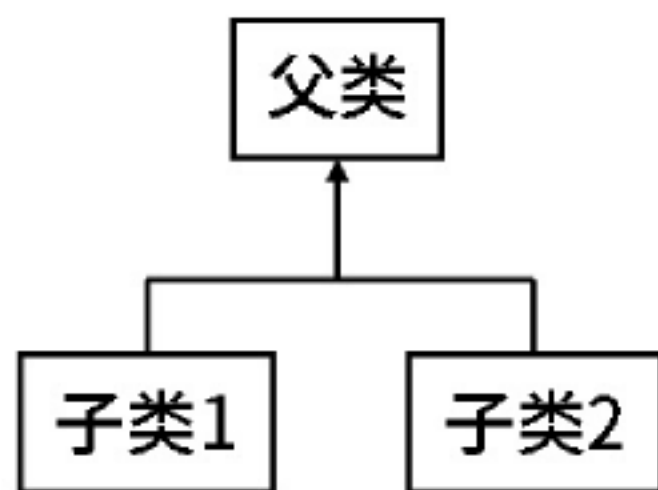
例：学员类、课程类



面向对象设计原则



- ★ 单一职责原则：设计**目的单一**的类
- ★ 开放-封闭原则：对扩展开放，对**修改封闭**
- ★ 李氏（Liskov）替换原则：**子类可以替换父类**
- ★ 依赖倒置原则：要依赖于抽象，而不是具体实现；**针对接口编程**，不要针对实现编程
- ★ 接口隔离原则：使用多个专门的接口比使用单一的总接口要好
- ★ 组合重用原则：要**尽量使用组合**，而不是继承关系达到重用目的
- ★ 迪米特（Demeter）原则（最少知识原则）：一个对象应当对其他对象有**尽可能少的了解**





习题讲解



最少知识原则（也称为迪米特原则）是面向对象设计原则之一，指一个软件实体应当尽可能少地与其他实体发生相互作用。这样，当一个实体被修改时，就会尽可能少地影响其他的实体。下列叙述中，“（ ）”不符合最少知识原则。

- A 在类的划分上，应当尽量创建松耦合的类
- B 在类的设计上，只要有可能，一个类型应当设计成不变类
- C 在类的结构设计上，每个类都应当尽可能提高对其属性和方法的访问权限
- D 在对其他类的引用上，一个对象对其他对象的引用应当降到最低



软件测试



- ✓ 软件测试方法 (★★)
- ✓ 软件调试 (★)



软件测试类型



动态测试
【计算机运行】

静态测试
【人工监测和计算机辅助分析】

白盒测试法

灰盒测试法

黑盒测试法

关注内部**结构与逻辑**

介于两者之间

关注**输入输出及功能**

桌前检查

代码审查

代码走查

都是做的 **【静态分析】**

自动化测试

先写脚本 -> 自动化执行

不适合场景：项目周期短，
需求变动频繁

控制流分析：是否存在没有使用的语句/**无法达到的语句**/调用并不存在的子程序。

数据流分析：引用**未定义的变量**、对以前未使用的变量再次赋值。

接口分析：模块之间**接口的一致性**、子程序和函数之间的接口一致性、函数形参与实参的数量、顺序、类型的一致性。

表达式分析：**括号不配对**、数组引用越界、除数为零。



白盒测试与黑盒测试

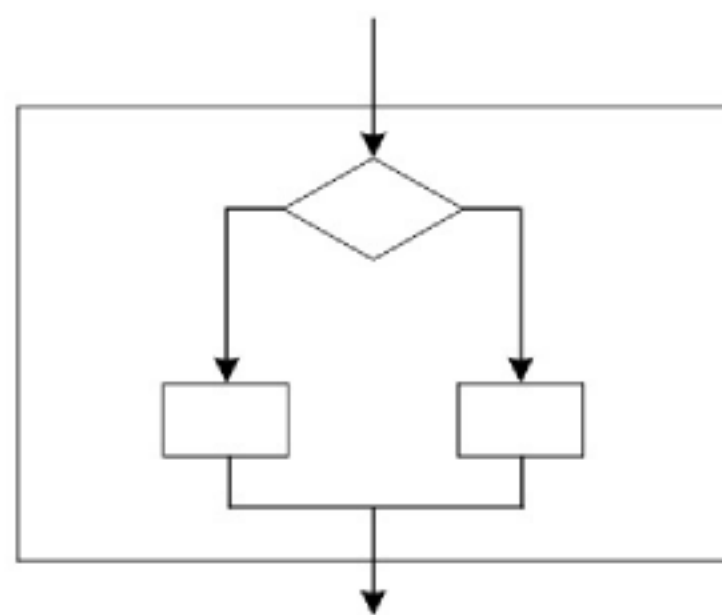


白盒测试【结构测试】：主要用于单元测试阶段。

控制流测试【逻辑覆盖测试（语句覆盖最弱，路径测试覆盖最强）】

数据流测试

程序变异测试【错误驱动测试】



黑盒测试【功能测试】：主要用于集成测试、确认测试和系统测试阶段。

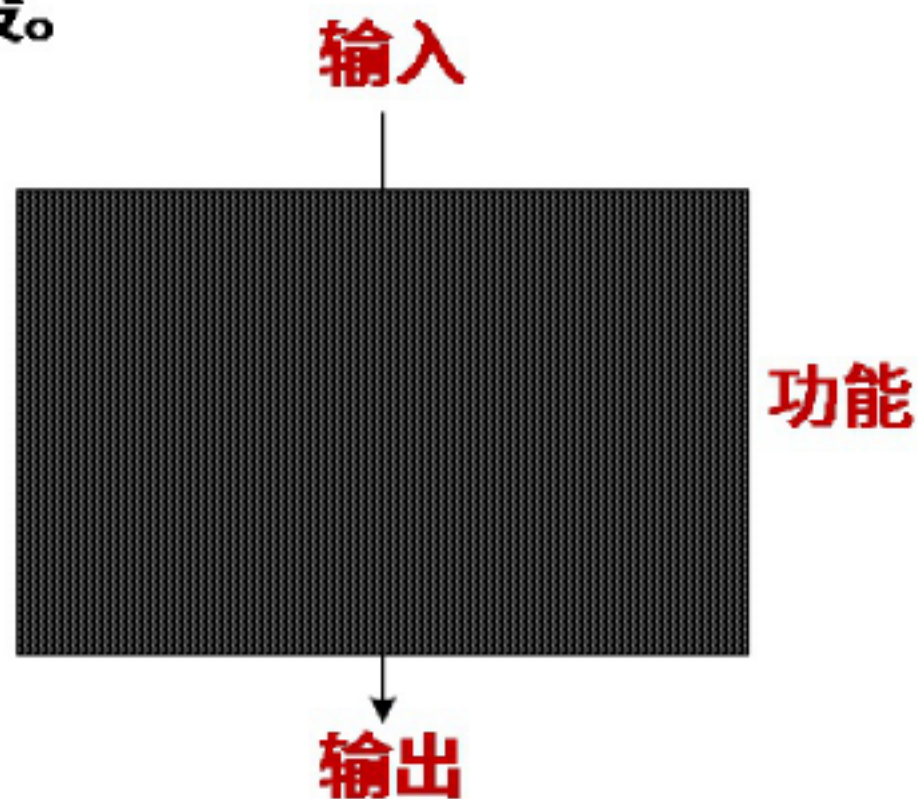
等价类划分：不同等价类，揭示不同问题；有效等价类/无效等价类。

边界值分析： $1 \leq x \leq 10$ ，可取x的值为0、1、10和11作为测试数据

错误推测：依靠测试人员的**经验和直觉**。

判定表：最适合描述在**多个逻辑条件取值**的组合所构成的复杂情况下，分别要执行哪些不同的动作。

因果图：根据**输入条件与输出结果之间的因果关系**来设计测试用例。





习题讲解



软件测试一般分为两个大类：动态测试和静态测试。前者通过运行程序发现错误，包括（ ）等方法；后者采用人工和计算机辅助静态分析的手段对程序进行检测，包括（ ）等方法。

A 边界值分析、逻辑覆盖、基本路径

B 桌面检查、逻辑覆盖、错误推测

C 桌面检查、代码审查、代码走查

D 错误推测、代码审查、基本路径

A 边界值分析、逻辑覆盖、基本路径

B 桌面检查、逻辑覆盖、错误推测

C 桌面检查、代码审查、代码走查

D 错误推测、代码审查、基本路径



习题讲解



在静态测试中，主要是对程序代码进行静态分析。“数据初始化、赋值或引用过程中的异常”属于静态分析中的（ ）。

- A 控制流分析
- B 数据流分析
- C 接口分析
- D 表达式分析



软件测试阶段



测试阶段

单元测试：依据【**详细设计**】，模块测试，模块功能、性能、接口等

集成测试：依据【**概要设计**】，模块间的接口

系统测试：依据【**需求文档**】，包括功能测试、性能测试、**验收测试**、压力测试等



确认测试：依据【**需求文档**】，验证软件与需求的一致性。
内部确认测试、Alpha测试、Beta测试、验收测试



软件测试阶段

其他测试	描述
AB测试	多版本同时使用，利于收集各版本的用户反馈，评估出最好版本。故也算是一种【网页优化方法】。
Web测试	Web系统测试与其他系统测试测试内容基本相同，只是测试重点不同。 Web代码测试包括：源代码规则分析、链接测试、框架测试、表格测试、图形测试等方面。
链接测试	链接测试可分为3个方面： 1、测试所有链接是否按指示的那样确实链接到了该链接的页面。 2、测试所链接的页面是否存在。 3、保证Web应用系统上没有孤立的页面。
表单测试	验证服务器是否能正确保存这些数据，后台运行的程序能否正确解释和使用这些信息。测试提交操作的完整性。
回归测试	测试软件变更之后，变更部分的正确性和对变更需求的符合性。



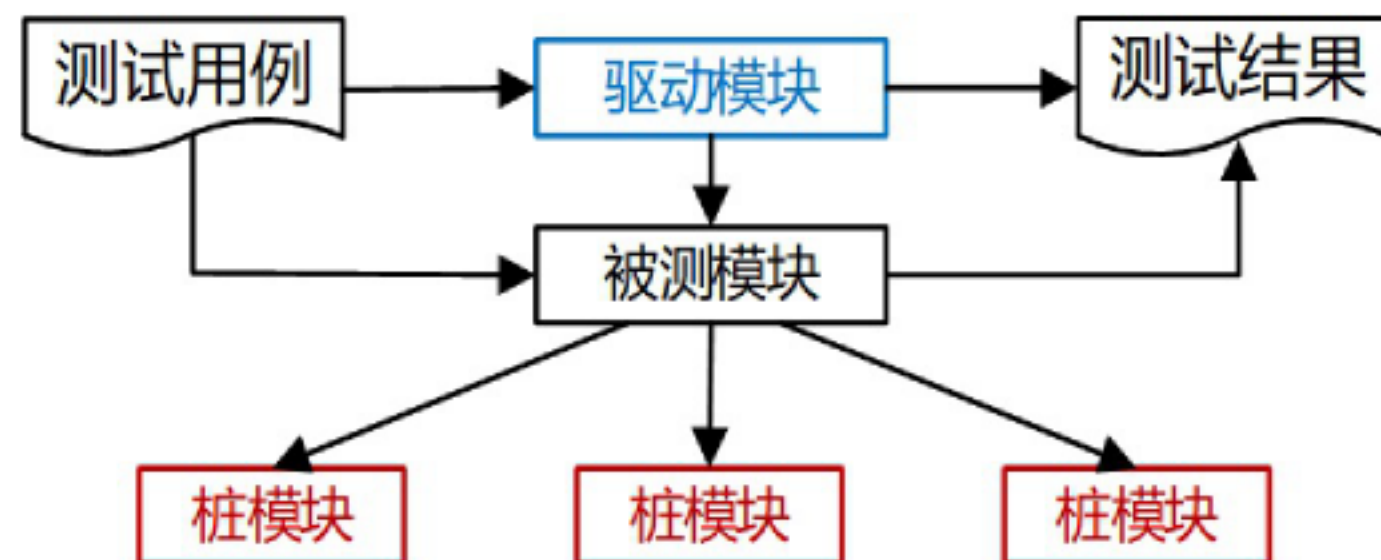
软件测试阶段



集成测试策略

- 一次性组装【风险高】
- 增量式组装【测试全面】

- ✓ 自顶向下【需要桩模块】
- ✓ 自底向上【需要驱动模块】
- ✓ 混合式【桩模块和驱动模块都需要】





软件系统测试



➤ 功能测试

➤ 性能测试

➤ 健壮性测试

➤ 用户界面测试

➤ 安全性测试

➤ 安装与反安装测试

- ✓ 负载测试：各种工作负载下系统的性能
- ✓ 压力测试【测上限】：系统的瓶颈或不能接受的性能点
- ✓ 强度测试【测下限】：系统资源特别低的情况下运行
- ✓ 容量测试【并发测试】：同时在线的最大用户数
- ✓ 可靠性测试：MTTF之类的参数



习题讲解



在单元测试中，（ ）。

- A 驱动模块用来调用被测模块，自顶向下的单元测试中不需要另外编写驱动模块
- B 桩模块用来模拟被测模块所调用的子模块，自顶向下的单元测试中不需要另外编写桩模块
- C 驱动模块用来模拟被测模块所调用的子模块，自底向上的单元测试中不需要另外编写驱动模块
- D 桩模块用来调用被测模块，自底向上的单元测试中不需要另外编写桩模块



习题讲解



软件性能测试有多种不同类型的测试方法，其中，（ ）用于测试在限定的系统下考查软件系统极限运行的情况，（ ）可用于测试系统同时处理的在线最大用户数量。

A 强度测试

B 负载测试

C 压力测试

D 容量测试

A 强度测试

B 负载测试

C 压力测试

D 容量测试



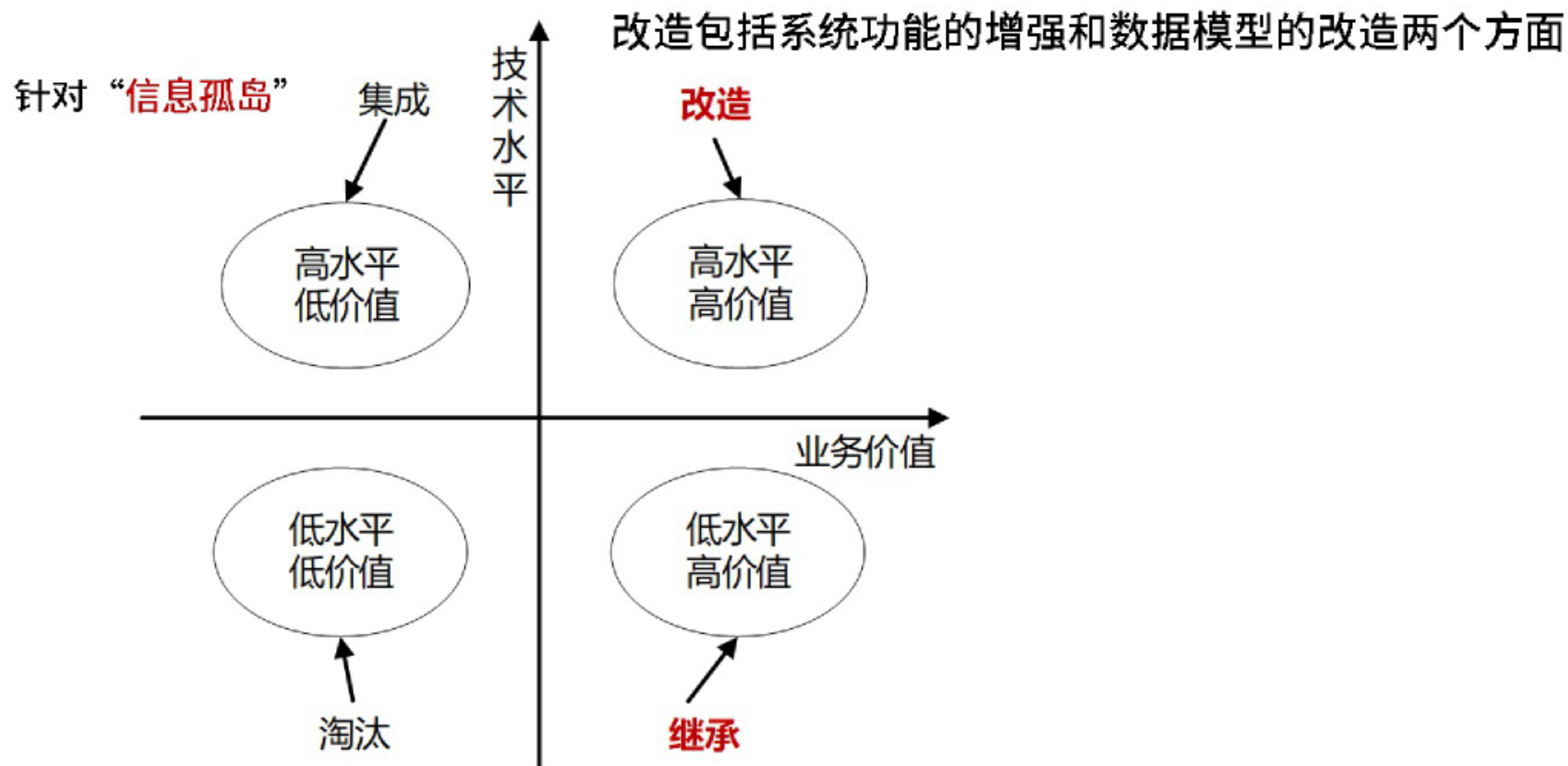
系统运行与软件维护



- ✓ 遗留系统与系统转换计划 (★★)
- ✓ 软件维护 (★★)



遗留系统演化策略



开发新系统时，需要完全兼容遗留系统的功能模型和数据模型



习题讲解



遗留系统的演化可以采用淘汰、继承、改造和集成四种策略。若企业中的遗留系统技术含量较高，业务价值较低，在局部领域中工作良好，形成了一个信息孤岛时，适合于采用（ ）演化策略。

A 淘汰

B 继承

C 改造

D 集成



新旧系统的转换策略

现有系统

新系统

直接转换策略

现有系统

新系统

并行转换策略

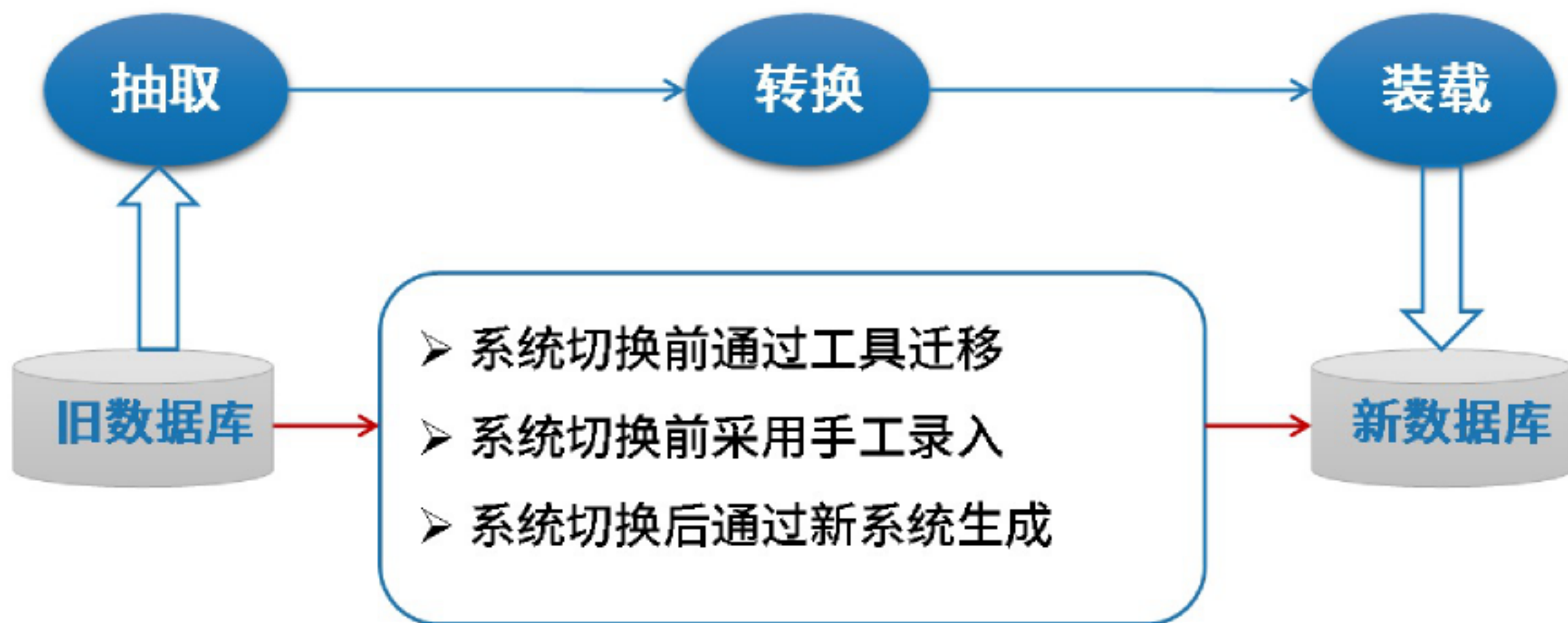
现有系统

新系统

分段转换策略



数据转换与迁移





影响软件可维护性的因素



【可理解性】 是指通过阅读源代码和相关文档，了解软件的功能和如何运行的容易程度。

【可修改性】 是指修改软件的难易程度。

【可测试性】 是指验证软件程序正确的难易程度。

可测试性好的软件，通常意味着软件设计简单、复杂性低。因为软件的复杂性越大，测试的难度也就越大。

【可靠性】 一个软件的可靠性越高，需要维护的概率就会越低。

【可移植性】 是指将软件从一个环境移植到新的环境下正确运行的难易程度。

软件运行环境的变化是软件维护的一种常见情形，可移植性好的软件会降低维护的概率。



软件维护类型



- ★ **正确性维护【修BUG】**：识别和纠正软件错误/缺陷，测试不可能发现所有错误。
- ★ **适应性维护【应变】**：指使应用软件**适应环境变化**【外部环境、数据环境】而进行的修改。
- ★ **完善性维护【新需求】**：扩充功能和改善性能而进行的修改。
- ★ **预防性维护【针对未来】**：为了**适应未来**的软硬件环境的变化，应主动增加预防性的新的功能，以使系统适应各类变化而不被淘汰。经典实例：**【专用】改【通用】**。



习题讲解



软件的维护并不只是修正错误。为了满足用户提出的增加新功能、修改现有功能以及一般性的改进要求和建议，需要进行（ ），它是软件维护工作的主要部分；软件测试不可能揭露旧系统中所有潜在的错误，所以这些程序在使用过程中还可能发生错误，诊断和更正这些错误的过程称为（ ）；为了改进软件未来的可维护性或可靠性，或者为了给未来的改进提供更好的基础而对软件进行修改，这类活动称为（ ）。

A. 完善性维护

B. 适应性维护

C. 预防性维护

D. 改正性维护

A. 完善性维护

B. 适应性维护

C. 预防性维护

D. 改正性维护

A. 完善性维护

B. 适应性维护

C. 预防性维护

D. 改正性维护